Kantonsschule Freudenberg

Maturitätsarbeit

Eine Drohne lernt sehen

Entwicklung eines Embedded Systems zur Echtzeit-Bildklassifizierung und Gestensteuerung

Leon Kaffarnik, 6c

Betreut von
Patric Rousselot

9. Dezember 2025

0.1 Abstract

Im Rahmen dieser Maturitätsarbeit wurde die Entwicklung einer autonomen Drohne, die mittels Künstlicher Intelligenz (KI) zur Bildverarbeitung auf visuelle Ereignisse (speziell Gesten) reagiert, untersucht und dokumentiert. Dazu wurden zwei Prototypen konzipiert: Der erste diente der Erarbeitung grundlegender Konstruktions- und Flugprinzipien. Der zweite Prototyp, gesteuert von einem eingebetteten Computersystem, erfasste und verarbeitete visuelle Daten mittels einer Kamera. Die Steuerung der Drohne basiert auf einer Bildklassifizierung durch ein KI-Modell. Es konnte gezeigt werden, dass eine Gestensteuerung über das eingebettete System in Echtzeit realisierbar ist.

0.2 Vorwort

„Ein Vogel fliegt gemäss der Prinzipien der Mathematik“
Leonardo Da Vinci, *Sul volo degli Uccelli, 1505*

Das Zitat von Leonardo Da Vinci fand sich in Tennekes Buch „The Simple Science of Flight“ [1]. Bereits Da Vinci war fasziniert von der Ähnlichkeit von Vögeln mit technischen und mathematischen Prinzipien. Historisch gesehen finden Drohnen, Helikopter und Flächenflugzeuge ihre Inspiration gleichermassen in der Natur. Allerdings unterscheiden sich die ersten, mit Leinen bespannten Flugobjekte und ebenso die heutigen Aluminiumriesen stark vom biologischen Vorbild. Ein Vogel kann, was moderne Flugzeuge nicht können; er kann denken. Dies zeigt sich beispielsweise in seiner Fähigkeit, eine Maus im Feld zu erkennen und eigenständig zu entscheiden, wie und ob er diese jagen wird. Die Übertragung dieser visuellen und kognitiven Fähigkeiten auf eine Drohne, basierend auf Künstlicher Intelligenz, bildet den Fokus dieser Maturitätsarbeit.

0.3 Danksagung

Für die umfassende Unterstützung und die wertvollen Beiträge im Rahmen dieses Projekts möchte ich meinen aufrichtigen Dank aussprechen. Die Realisierung dieser Arbeit wäre ohne das Engagement der nachfolgend alphabetisch aufgeführten Personen nicht möglich gewesen:

- Maurus Davatz: Für Erstellen der Bilddokumentation bei Testflügen.
- Michael Gloor: Für die Durchführung eines Interviews und einer Vorführung.
- Ute Kaffarnik: Für die finanzielle Unterstützung, das Lektorat und die Mitarbeit bei den Flugtests.
- Dr. Adrian Kohlbrenner: Für die Finanzielle Unterstützung, das Lektorat und die Mitarbeit an unterschiedlichen Tests.
- Prof. Dr. Teddy Loeliger: Für die Durchführung eines Interviews und einer Vorführung.
- Patric Rousselot: Für die fachliche Betreuung und das Lektorat.

Inhaltsverzeichnis

0.1 Abstract ii
0.2 Vorwort ii
0.3 Danksagung ii

Abbildungsverzeichnis **iv**

Tabellenverzeichnis **v**

Glossar **vii**

1 Einleitung **1**
1.1 KI - Hintergrund zur Entstehung 1
1.2 Fragenstellung und Themengebiete 3
1.3 Aufbau der Arbeit 4

2 Aktuelle Forschung und Applikation **5**
2.1 Verity 5
2.2 ZHAW 6

3 Einführung in die Drohnentechnologie **7**
3.1 Methodik 7
3.2 Aufbau einer Drohne 8

4 Gestenerkennung **12**
4.1 Einführung 12
4.2 Methodik 12
4.3 Projekte 13

5 Konstruktion und Basis-Hardware der KI-Drohne **18**
5.1 Einführung 18
5.2 Methodik 18
5.3 Übersicht 19
5.4 Bau 20
5.5 Rechtliches im Umgang mit Drohen 22

6 Systemintegration und Steuerungsdesign **24**
6.1 Einführung 24

6.2 Verbinden des Companion Computers mit dem Flight Controller 24
6.3 Schreiben von Python-Skripten 26
6.4 Flug kontrolliert durch den CC 29
6.5 Gestensteuerung auf dem RPI 30

7 Ergebnisse **33**
7.1 Minimalistische Drohne 33
7.2 Eckdaten: KI-Drohne 34
7.3 Generelles Flugverhalten 34
7.4 Testsoftware 35
7.5 Testverfahren 35
7.6 Zuverlässigkeit der Handerkennung 36
7.7 Charakterisierung der Verbindung 37
7.8 Modifikation der Drohne 37

8 Diskussion **38**
8.1 Erreichen der Ziele 38
8.2 Probleme, Rückschläge und Gelerntes 39
8.3 Flugverhalten der ersten Drohne 42
8.4 Ausblick 42

Verwendung von KI **47**

Abbildungsverzeichnis

1.1 Perzeptron [3]. 2

2.1 Verity Drohne beim Flug durch ein Lagerhaus [9]. 5
2.2 Forschungsdrohne der ZHAW mit TOF-Kameramodulen [14]. 6
2.3 TOF-Kamera der ZHAW [14]. 6

3.1 Minimalistische Drohne. 7
3.2 Arm der Drohne mit ESC, Motor und Propeller. 9
3.3 PWM-Signal [19]. 9
3.4 FC in Grün, Barometer in Violett und IMU in Blau. 10
3.5 Rotationsmessung mit (blau) und ohne Filter (rot) [16]. 10
3.6 Vereinfachte Fluglageregelung einer Drohne. 11

4.1 Template-Matching durch einen mathematischen Suchalgorithmus (TM_CCOEFF) [29]. 13
4.2 MP Gestenerkennung. 14
4.3 Unterschiedliche Schritte der Gestenerkennung. 15

5.1 Übersicht über den Informationsfluss der Drohne ohne CC. 19
5.2 Drohne aus verschiedenen Perspektiven. 20
5.3 Telemetrieradio. 21
5.4 Kalibrierung der Drohne. 21
5.5 Befestigte Drohne vor dem Lauftest. 22
5.6 Drohne beim ersten Testflug. 23

6.1 CLI des RPI bei Verbindung mit dem FC mittels MavSDK. 25
6.2 Mediapipe auf dem RPI. 30
6.3 Logik des Computerprogramms im RPI. 31

7.1 Grenzen von Mediapipe beim Zeigen eines Handrückens. 36

8.1 Aufladen eines tiefenentladenen Akkus. 40
8.2 Erster Testflug. 42

Tabellenverzeichnis

3.1 Komponenten und Funktionen - Bauteile der minimalistischen Drohne. . . . 8

5.1 Hardware- und Software-Komponenten nebeneinander. 18

7.1 Eckdaten der minimalistischen Drohne. 33
7.2 Eckdaten der KI-Drohne. 34

Glossar

ABKÜRZUNG	ERLÄUTERUNG
AP	ArduPilot ist eine Open Source Firmware.
Arming	Beim Arming wird die Drohne angelassen. Ähnlich wie beim Auto drehen sich danach die Propeller im Leerlauf. Damit man die Drohne armen kann, muss ein sogenanntes Prearming zuvor erfolgreich sein. Das ist ein automatisierter Sicherheitscheck, der beispielsweise die GPS-Verbindung und alle Sicherheitsmechanismen überprüft.
CC	Der Companion Computer, ist ein Computer, welcher mit dem FC kommuniziert und Kommandos gibt, um die Drohne zu bewegen. Dabei gibt er dem FC eine Route (Geschwindigkeiten, Positionen, etc.) vor.
DroneKit	DroneKit ist eine Python-Library, um MAVLink Kommandos zu übermitteln.
ESC	Die Engine Speed Controller sind dafür verantwortlich, die bürstenlosen Motoren anzusteuern. Die gewünschte Drehgeschwindigkeit wird dabei mit einem PWM-Signal vom FC vorgegeben.
FC	Flight Controller ist die zentrale Steuerungseinheit und übernimmt die Aufgabe der Regulierung der Drohne. Dabei werden Sensordaten eingelesen und entsprechende Output Signale an die ESCs (mit PWM-Signalen) geschickt. Der FC hält eine Drohne also stabil in der Luft.
FPS	Frames per second (Bilder pro Sekunde) ist die Bildwiederholrate.
GPS	Durch das Global Positioning System lässt sich die Position einer GPS-Antenne sehr genau bestimmen. Die Genauigkeit reicht, um eine Drohne zu stabilisieren.
IMU	Die Inertial Measurement Unit misst Linearbeschleunigungen und Drehraten in alle Richtungen.
MAVLink	MAVLink ist ein Kommunikationsprotokoll, um mit Drohnen zu kommunizieren.
MP	MediaPipe: Open-Source-Framework von Google für Anwendungen von Maschinellem Lernen (hier: Hand-Landmarken).

OpenCV	OpenCV ist eine Computer-Vision Library. Sie übernimmt das Aufnehmen, Formatieren, Bearbeiten und Beschriften von Bildern sowie Bewegtbildaufnahmen und kann auch Bildklassifizierung betreiben.
PID	Ein PID-Regler ist ein Regelungsmechanismus. Damit lassen sich Systeme regeln, die Trägheit und Verzögerung haben, wie zum Beispiel eine Drohne.
PWM	PWM oder Pulsweitenmodulation ist ein binäres Signal, das zur digitalen Steuerung von Analoggeräten (wie ESCs) verwendet wird, indem das Verhältnis zwischen der Einschaltdauer und der Gesamtperiode (Tastverhältnis) variiert wird.
QGC	QGroundControl ist die Software, die auf dem Laptop läuft und via Telemetrieradios mit dem FC kommuniziert. Über QGC werden hier vor allem Einstellungen am FC verändert. Es ist eine Zugangssoftware zum FC.
RC	RC steht für Radio Control. Für die RC-Verbindung wird meist eine Fernbedienung, also ein RC-Controller, und ein Empfänger an der Drohne, ein RC-Receiver verwendet.
RPI	Der Raspberry PI ist ein kleiner, leichter, Linux-basierter Computer, der bei diesem Projekt für die Bildverarbeitung und Steuerung der Drohne verwendet wird.
SBUS	SBUS oder Serial-Bus ist ein Kommunikationsprotokoll und wird verwendet, um mehrere Steuersignale über eine einzige Leitung zu übertragen.
SSH	Die Secure Shell erlaubt den Zugriff bzw. die Steuerung eines Computers ohne physische Verbindung. Dabei wird die Verbindung über ein Netzwerk geleitet und erlaubt ein CLI, während dem Flug, des RPIs auf einem Laptop.
UART	UART ist eine Schnittstelle, um serielle Datenströme zu übertragen.
Venv	Ein Virual Environment ist eine Umgebung, in welche man (Python) Pakete installiert und das man beim Ausführen eines Programmes aktiviert.

Kapitel 1

Einleitung

Eine Drohne ist ein unbemanntes Luftfahrzeug (UAV), also ein Luftfahrzeug ohne Besatzung. Drohnen haben im Gegensatz zu Menschen entscheidende Vorteile, denn sie können an Orte gelangen, die für Menschen schlecht zugänglich sind, weiter können sie Risiken in Kauf nehmen, ohne das Leben von Piloten zu gefährden. Deshalb sind sie prädisponiert, um gefährliche Arbeit in unzugänglichem Gelände auszurichten.

Beispielsweise benutzt die Rega (Schweizerische Rettungsflugwacht) Drohnen zur Personensuche [2]. Diese können auch bei schlechter Witterung fliegen. Sie suchen ein Mobilfunksignal und lokalisieren damit Vermisste. Während die Rega Drohne primär Mobilfunksignale zur Lokalisierung nutzt, können KI-gesteuerte Drohnen eine zusätzliche oder alternative Suchmethode darstellen, indem sie Vermisste direkt mittels Bilderkennung erkennen.

Drohnen mit KI haben Vorteile gegenüber Flugzeugen, Helikopter und Drohnen ohne KI. Denn sie können durchgehend effizient arbeiten, unterliegen keiner Ermüdung und reduzieren das Unfallrisiko für ihre Piloten, wodurch sie ihre Aufgabe dauerhaft mit hoher Präzision erfüllen. KI-gesteuerte Drohnen sind auch deshalb für die Personensuche ideal.

Im Kontext dieser Arbeit wird die Computer Vision in der Folge zur Vereinfachung oftmals nur als KI bezeichnet, da andere KI-Ansätze nicht thematisiert werden. Bei Computer Vision geht es darum, visuelle Informationen aus der realen Welt (Bilder oder Videos) zu verarbeiten, zu analysieren und zu verstehen, um auf dieser Grundlage automatisierte Entscheidungen oder Handlungen auszuführen.

1.1 KI - Hintergrund zur Entstehung

Doch KI ist keine neue Erscheinung. Es gab bereits im Jahr 1957 Perzeptronen. Das sind künstliche Neuronen, welche entscheiden, ob sie ein Signal abgeben (siehe Abbildung 1.1) [3]. Solche künstlichen Neuronen sind auch heute noch die Grundbausteine der KI. Gegen Ende der 60er Jahre wurde klar, dass Perzeptronen zwar einige logische Probleme lösen können, jedoch nicht alle. Die Fähigkeit der Perzeptronen beschränkte sich auf die Erkennung simpler geometrischer Figuren; die komplexe Mustererkennung, wie die Detektion einer menschlichen Hand, überstieg die Möglichkeiten dieser frühen Architektur deutlich.

Die Forschung machte erst in den 80er Jahren Fortschritte, als mehrere Perzeptronen hintereinandergeschaltet wurden, wodurch mehrschichtige Architekturen entstanden [4, 5]. In den 90er Jahren konnte man mit solchen KI-Modellen schon deutlich mehr machen. Bei-

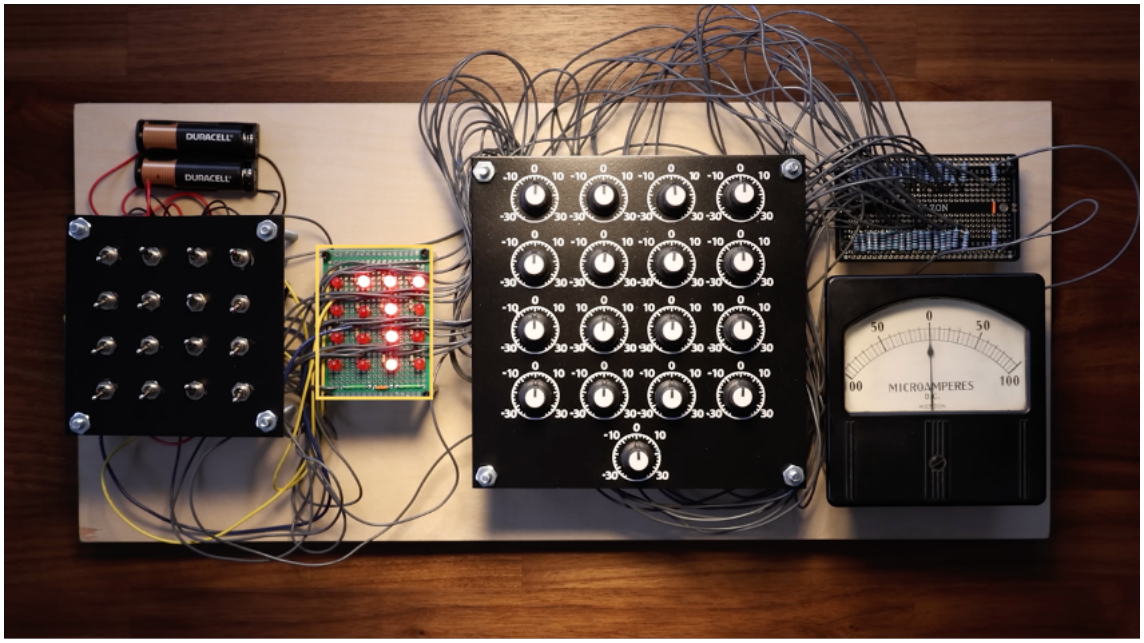


Abbildung 1.1: Perzeptron [3].

spielsweise entstand mit LeNet-5 ein Modell, mit dem sowohl Hand- als auch Druckschriften in einem Bild erkennt werden konnten [6].

Solche KI-Modelle, die bei unterschiedlichen Formen und Konturen beurteilen, was diese darstellen, nennt man bildklassifizierende KI [7]. Die bildklassifizierende KI ist eine Form von Computer Vision [8]. Dabei kann der Output beispielsweise eine Wahrscheinlichkeit sein, mit der das Input-Bild eine Hand enthält, oder es können auch Bildkoordinaten der Hand sein.

Solche Modelle umgeben uns bereits heute in unserem Alltag. Heute erkennt ein Auto mit Hilfe von Computer Vision die Mittellinie der Strasse und auch Verkehrsschilder. Diese Fähigkeit ist für ein selbstfahrendes Auto unabdingbar. Neben vielen weiteren Anwendungen kann man mit Hilfe von KI auch Flugzeuge und Drohnen steuern.

1.2 Fragenstellung und Themengebiete

Bei dieser Arbeit geht es um die Implementierung einer Gestensteuerung auf einer Drohne und den Bau einer solchen Drohne. Das Ziel ist, zu ergründen, wie man eine Drohne mittels KI steuern und auf visuelle Ereignisse reagieren lassen kann. Kernstück des Projekts ist die Erstellung einer Plattform, die als Basis für die Implementierung verschiedener KI-Anwendungen dienen kann.

Themengebiete

Hauptsächliche Themengebiete sind:

1. Grundsätzlicher Aufbau von Drohnen.
2. Regelungstechnik (PID-Kontroller).
3. Elektrotechnik (Zusammenstellen von Kompatiblen Bauteilen, Löten, verstehen von Protokollen und Schnittstellen).
4. Legale Vorgaben bei Drohnenflügen.
5. Verbindung und automatisierte Kommunikation bei Embedded Systemen (spezialisierter Computer, um in einem System eine bestimmte Aufgabe zu übernehmen).
6. Sicherheit und Redundanz bei Python-Skripten.
7. Hand-/Gestenerkennung mit KI.

Fragen

Nachfolgend zu ergründende Fragen.

1. Wie funktioniert Gestenerkennung grundsätzlich?
2. Wie optimiert man KI für Computer mit wenig Rechenleistung?
3. Wie minimiert man das Unfallrisiko beim Entwickeln einer Drohne?
4. Wie funktioniert die Kommunikation und deren Automation bei den unterschiedlichen Bauteilen?
5. Wie verarbeitet man ein Bild in einen Befehl, um die Drohne zu bewegen?
6. Lässt sich die Drohne somit mit Gesten steuern?

1.3 Aufbau der Arbeit

Diese Arbeit sollte mit einem Grundwissen von Informatik, Technik und Physik verstanden werden können. Es werden grundsätzliche Begriffe und Konzepte erläutert, welche über den Grundkenntnissen von Maturanden liegen und für das Verständnis der Arbeit zentral sind. Die Entwicklung zweier Drohnen steht im Mittelpunkt der Arbeit: Die minimalistische Drohne ist so simpel gebaut wie möglich. Die KI-Drohne ist komplexer, mit mehr Redundanz, besseren und mehr Sensoren. In der vorliegenden Arbeit werden folgende Themen in chronologischer Reihenfolge besprochen:

1. Anhand von zwei Beispielen wird veranschaulicht, wie KI-gesteuerte Drohnen genutzt werden und woran geforscht wird.
2. Mithilfe der minimalistischen Drohne wird erläutert, aus welchen Bauteilen eine Drohne grundsätzlich aufgebaut ist und wie diese zusammen funktionieren.
3. Computer Vision wird eingeführt und Hand- beziehungsweise Gestenerkennung erklärt und erstellt.
4. Die KI-Drohne wird zusammengestellt, aufgebaut und geflogen.
5. Der RPI wird mit dem Flugkontroller der Drohne verbunden und die Verbindung automatisiert.
6. Mittels eines Python-Skripts wird die Drohne gesteuert.
7. Es werden das Produkt- sowie Testergebnisse vorgestellt.
8. Über die Ziele und den Arbeitsprozess wird reflektiert.

Ausserdem gibt es ein Glossar, um Abkürzungen sowie wichtige Konzepte und Bauteile nachschlagen zu können. Zwei Methodikkapitel sind bei den jeweiligen Drohnen zu finden.

Kapitel 2

Aktuelle Forschung und Applikation

Mit dem Beispiel der Rega wurde eine Anwendung im Bereich der Personensuche gezeigt. Es gibt aber mehr Anwendungen für Drohnen mit KI. Besprochen werden ein Beispiel der Industrie und eines der Forschung [1].

2.1 Verity



Abbildung 2.1: Verity Drohne beim Flug durch ein Lagerhaus [9].

Die Firma Verity entwickelt und baut Drohnen, die für Lichtshows genutzt werden, sowie Drohnen zur Überprüfung von Lagerinventar (Abbildung 2.1). Die Firma beschreibt ihre Geschichte folgendermassen [9]:

„Verity was launched in 2014 out of ETH Zurich,[...]. In 2016, Verity emerged by stealth, achieving what others wouldn't dare—flying drones over a TED audience, [...]. Today, Verity's systems transform businesses across industries—enabling zero-error warehouses through automated inventory tracking, analytics, and process optimization, [...].“

[1] KI in Konsumentendrohnen wird nicht thematisiert.

Michael Gloor arbeitet als *head of platform* bei Verity [10]. Er erklärte, dass Verity ihre Software auf eigenen Drohnen abspiele.

2.2 ZHAW

Eine andere, akademische Anwendung lässt sich am Beispiel der ZHAW (Zürcher Hochschule für angewandte Wissenschaften) aufzeigen. Im Rahmen eines Forschungsprojekts wurde dort eine Drohne mit Gestensteuerung, Objektverfolgung und Objektausweichung umgesetzt (siehe Abbildung 2.2) [11, 12].

Dabei wurde nicht mit einer RGB-Kamera gearbeitet, sondern mit einer TOF-Kamera (Time of Flight) [13]. Diese misst die Zeit, welche selbst ausgestrahltes Licht (meist Infrarot) braucht, um von der Umgebung reflektiert zu werden und zurückzukommen. Dadurch lässt sich ein Tiefenbild erstellen, durch welches Distanzen dreidimensional messbar werden.

Prof. Dr. Teddy Loeliger erklärte, dass man mit dem Tiefenbild, welches die TOF-Kameras aufnehmen, nahe Objekte (wie eine Hand) vom Hintergrund unterscheiden und separieren könne [14]. Vom Seitenverhältnis dieses Objektes liesse sich dann auf eine Aktion schliessen (siehe Abbildung 2.3) [15].



Abbildung 2.2: Forschungsdrohne der ZHAW mit TOF-Kameramodulen [14].

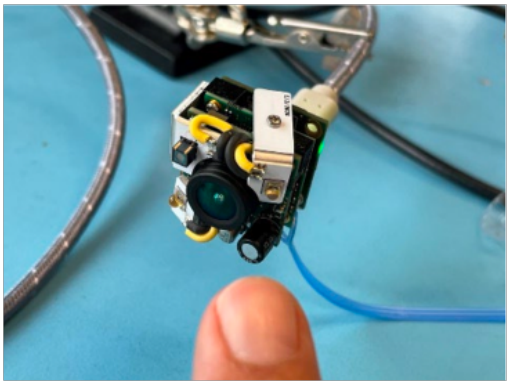


Abbildung 2.3: TOF-Kamera der ZHAW [14].

Kapitel 3

Einführung in die Drohnentechnologie

Erst wurde eine Drohne gebaut, die simpel ist, um den grundsätzlichen Aufbau von Drohnen zu verstehen. Diese *minimalistische Drohne* hat alles, was eine Drohne grundsätzlich zum Fliegen braucht (siehe Abbildung 3.1). In diesem Kapitel wird zunächst erklärt, wie eine Drohne aufgebaut ist und wie sie funktioniert.

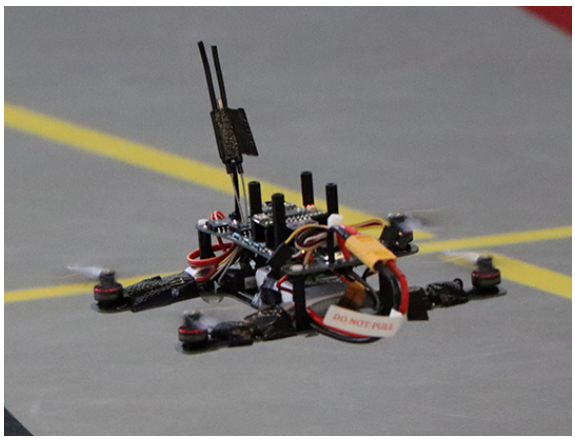


Abbildung 3.1: Minimalistische Drohne.

3.1 Methodik

Für das Vorprojekt wurde das Projekt von Laurens Raes übernommen [16]. Für die Hardware wurden, je nach Verfügbarkeit, gleiche oder ähnliche Teile bestellt, wobei stets auf Kompatibilitätsprobleme zu achten ist (siehe Tabelle 3.1). Der FC besteht aus einem Mikrokontroller, der selbst programmiert werden kann.

KOMPONENTE	FUNKTION
TEENSY 4.0	FC
GY-521 MPU-6050	Gyroskop
GY-BMP280	Barometer
GEPRC GR1105 5000 KV	Motor
HOBBYKING 6A ESC WITH BEC	ESC
GEMFAN 3018R	Propeller (Uhrzeigersinn)
GEMFAN 3018	Propeller (Gegenuhrzeigersinn)
TURNIGY 2S 1300 MAH	Batterie
HOBBYKING B3AC	Batterieladegerät
FLYSKY FS-I6	Transmitter
FLYSKY FS-IA6B	Receiver
NICHT KOMMERZIELL VERFÜGBAR	PCB

Tabelle 3.1: Komponenten und Funktionen - Bauteile der minimalistischen Drohne.

3.2 Aufbau einer Drohne

ESCs und Motoren

Im Schwebeflug entspricht die Auftriebskraft $F_{AUFTRIEB}$ ungefähr der Gravitationskraft $F_{Gravitation}$. Diese Auftriebskraft wird durch das ständige Beschleunigen von Luft nach unten erzeugt. Dazu hat eine Drohne Propeller (siehe Abbildung 3.2), die durch eine Drehbewegung Luft nach unten beschleunigen. Um die Propeller zu drehen, braucht es Motoren. Bei Drohnen sind diese meist bürstenlos (Brushless DC) [17]. Bürstenlose Motoren anzutreiben ist komplex. Es braucht einen sogenannten Engine Speed Controller (ESC) [18]. Um einen ESC zu steuern, braucht es ein PWM-Signal (mit kleinem Strom und Spannung) und Strom [19]. Letzterer wird von der Batterie gespiesen.

PWM oder Pulsweitenmodulation besteht aus einem binären Signal, welches sich ein- und ausschaltet. Der Abstand zwischen dem Einschalten und dem nächsten Einschalten (Frequenz) bleibt gleich (zu sehen in Abbildung 3.3).

Batterie

Zur Energieversorgung dienen Lithium-Polymer-Akkumulatoren (LiPo-Akkus); alternativ sind auch andere Batterietypen möglich. Da einige Komponenten eine reduzierte und konstante Spannung benötigen, erfolgt die Anpassung typischerweise mittels Spannungsteilern oder Step-Down-Konvertern. Für die Leistung gilt:

$$P = U \cdot I$$

Man kann nun die Zellspannung U_{ZELLE} also Spannung pro Zelle) mit der Zellenanzahl multiplizieren – da die Zellen hintereinander geschalten sind [2]. Es gilt:

$$U = U_{ZELLE} \cdot S$$

[2]Hier wird mit der maximalen Spannung und dem maximalen Strom für die Dauerleistung gerechnet.



Abbildung 3.2: Arm der Drohne mit ESC, Motor und Propeller.

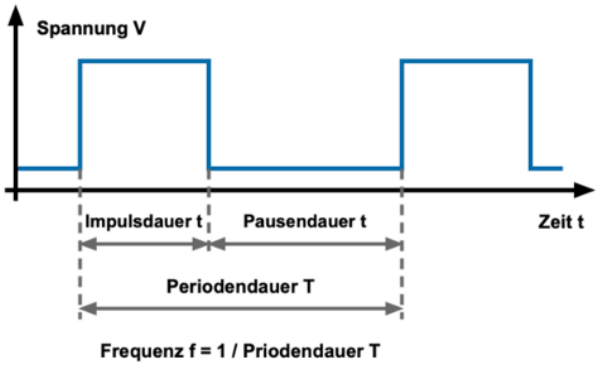


Abbildung 3.3: PWM-Signal [19].

Bei Batterien gibt es einen Multiplikationsfaktor (in C_{RATING} angegeben) für den gilt wobei C die Kapazität ist:

$$I_{MAX} = C \cdot C_{RATING}$$

Für die Batterie gilt konkret; $U_{ZELLE} = 4.2V, S = 2, C = 1.3Ah, C_{RATING} = 30$. Also ist die maximale Dauerleistung:

$$P_{MAX} \approx 330W$$

Sicherheit beim Umgang mit Batterien

Lipos sind grundsätzlich sehr gefährlich, da sie alle Bedingungen erfüllen, um sich zu entzünden. Vor allem sind das:

- Wärme (durch den inneren Widerstand).
- Brennstoff (durch Lösungsmittel und Grafit-Anode).
- Oxidationsmittel (durch Metalloxide wie Lithium-Kobalt-Oxid an der Kathode).

Die Energiemenge ist dabei nicht nur im geladenen Zustand hoch, der Akku ist auch entladen oder sogar tiefenentladen noch sehr gefährlich.

Typischerweise sehen Unfälle so aus: Der Akku wird entweder physisch beschädigt (darunter zählt auch ein Fall) oder falsch geladen (meistens zu schnell). Dadurch gibt es entweder direkt einen Kurzschluss oder die chemische Struktur verändert sich (bei Beschädigung passiert das oft durch Luftfeuchtigkeit, ansonsten durch ein falsches Laden). Anschliessend explodiert, berstet oder brennt der Akku (oft passiert alles davon).

Hier ist wichtig zu beachten, dass der Akku bei Nichtbenutzung immer in einer Sicherheitstasche für Lipo-Batterien gelagert wird und nie unbeaufsichtigt ge- oder entladen wird.

Flight Controller (FC)

Der FC (siehe Abbildung 3.4) gibt den ESCs die PWM-Signale [20]. Um den Schub an jedem Propeller zu bestimmen, muss der FC wissen, wie er im Raum liegt, sich bewegt und sich bewegen soll. Dazu werden Informationen (PWM-Signale) von der Fernbedienung bzw. ihrem Empfänger (oder Receiver) genommen. Aus diesen Informationen kann er mittels eines PID-Reglers PWM-Werte an die ESCs geben, um die Drohne stabil in der Luft zu halten.

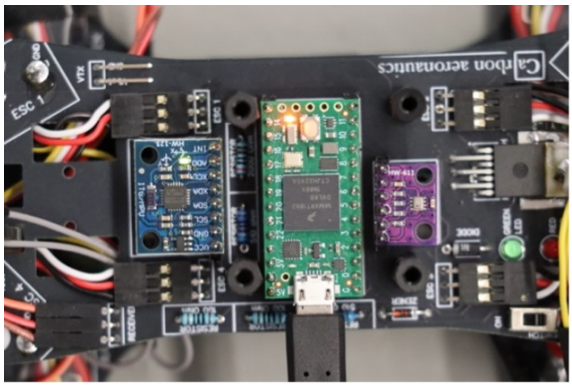


Abbildung 3.4: FC in Grün, Barometer in Violett und IMU in Blau.

IMU

Die *Inertial Measurement Unit* oder kurz IMU misst Linearbeschleunigungen und Drehraten in alle Richtungen [21]. Zum Beispiel misst sie 1G in positiv-vertikaler Richtung (z-Richtung), wenn die Drohne auf dem Tisch liegt.[3]

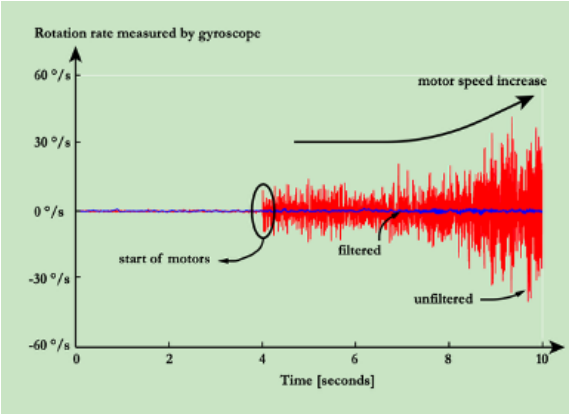


Abbildung 3.5: Rotationsmessung mit (blau) und ohne Filter (rot) [16].

Man sieht beispielsweise in Abbildung 3.5, wie einzelne Werte bei 30°/s sind, während

[3]Die Zahlen der IMU sind nicht schön und die IMU misst nicht perfekt, deshalb muss man sie jeweils noch kalibrieren.

die Drohne jedoch bei 0°/s ist, was an den Vibrationen liegt, die durch die Propeller induziert werden. Um von den roten Messwerten zu den Blauen zu kommen, müssen hohe Frequenzen herausgefiltert werden. Das geschieht üblicherweise durch *Tiefpassfilter* [22].

Steuerung

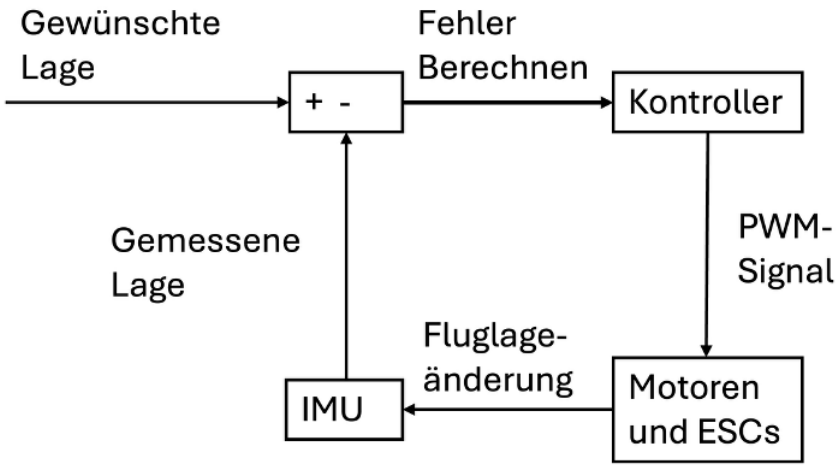


Abbildung 3.6: Vereinfachte Fluglageregelung einer Drohne.

In Abbildung 3.6 zu sehen ist ein stark vereinfachtes Flussdiagramm einer typischen Fluglageregelung einer Drohne. Der eigentliche Output ist hierbei die Fluglage. Bei dieser Drohne gibt es im Kasten *Kontroller* mehrere PID-Regler [23]. Ein PID-Regler ist ein simpler, aber effektiver Regelungsmechanismus. Damit lassen sich Systeme regeln, die Trägheit und Verzögerung haben wie eine Drohne. Mathematisch sieht der Regler folgendermassen aus:

$$y(t) = K_P \cdot e(t) + K_I \cdot \int_0^t e(\tau)\, d\tau + K_D \cdot \frac{de(t)}{dt}$$

Interessant sind vor allem die konstanten P-, I- und D-Werte bzw. K_P, K_I, K_D. Wenn diese (durch sogenanntes PID-Tuning) gut eingestellt sind, gibt diese Funktion Steuerungswerte, um die gewünschte Bewegung oder Orientierung zu erreichen.

Receiver und Fernbedienung

Um die Flugbahn mit einer Fernbedienung zu bestimmen, werden die Steuersignale, meist über Funk, an einen Receiver (Empfänger) mit Antenne geschickt. Dieser nimmt die Signale auf und gibt sie in einem PWM-Signal (oder digitalen Signal) an den FC weiter. Bei der KI-Drohne des Hauptprojekts wird der Receiver über eine SBUS-Schnittstelle mit dem FC kommunizieren [24]. SBUS oder Serial-Bus ist ein Kommunikationsprotokoll und wird verwendet, um mehrere Steuersignale über eine einzige Leitung zu übertragen.

Kapitel 4

Gestenerkennung

4.1 Einführung

Die KI-Drohne soll sich mittels Kamera und Bildklassifizierung durch KI autonom bewegen. Das bedeutet in der Praxis, dass die Kamera beispielsweise das Bild einer Faust bekommt und dieses an den Computer für die Bildverarbeitung weitergibt. Ein KI-Modell erkennt dort die Faust und interpretiert sie als direkten Bewegungsbefehl für die Drohne. Anschliessend soll sich die Drohne entsprechend bewegen.

Die Bildverarbeitung ist hier deutlich komplexer als bei der ZHAW-Drohne. Zum einen liegt das am 2-Dimensionalen Bild (statt dem 3-Dimensionalen Bild der ZHAW-Drohne), welches als Input verwendet wird, zum anderen wird nicht über ein einfach zugängliches Seitenverhältnis gearbeitet, sondern über eine viel kompliziertere Gestensteuerung. Dafür können mehr unterschiedliche Befehle durch die Gesten übermittelt werden.

4.2 Methodik

Als Companion Computer (CC) wird ein Raspberry PI (RPI) verwendet (mit der jeweils aktuellen Version des Raspberry Pi OS), als Laptop wird ein Macbook Air (kurz Mac) benutzt. Als IDE (Programmierumgebung) wird VSCode (Visual Studio Code von Microsoft) und Nano gebraucht. Als Computer Vision Library wird OpenCV und für die Handorientierungspunkteerkennung wird MediaPipe (kurz MP, von Google) inklusive der vortrainierten KI-Modelle verwendet [25, 26]. Alle diese Operationen werden für das Projekt benötigt [4]. Für die Verbindung zum FC wird DroneKit verwendet, eine Python Bibliothek für MAVLink [27].

[4] Was hierbei in der Wichtigkeit häufig unterschätzt wird, ist Community Support und gute Dokumentationen. Beide Faktoren sprechen stark für OpenCV und separieren es von ähnlichen Programmen. Für Projekte dieser Art ist es aus zeitlichen Gründen sehr wichtig, dass die einzelnen Funktionen der Library gut dokumentiert sind.

4.3 Projekte

Im Vorfeld wurden Tutorials zu OpenCV angeschaut und nach-programmiert [28]. OpenCV ermöglicht Farbkorrekturen, Ausschneiden eines Teilbildes und Zeichnen auf ein Bild [29]. Dazu kommen vortrainierte KI-Modelle, die ein Gesicht im Bild suchen und finden können, sowie mathematische Suchalgorithmen. Damit lassen sich zum Beispiel das Gesicht Messis in einer Totalaufnahme von einem Fussballspiel finden (siehe Abbildung 4.1). Links ein Gesicht, welches gesucht wird. In der Mitte ist die Übereinstimmung des linken Bildes mit dem Bildausschnitt eines anderen Bildes – je weisser, desto ähnlicher. Rechts ist der Rahmen am wahrscheinlichsten Ort.



Abbildung 4.1: Template-Matching durch einen mathematischen Suchalgorithmus (TM_CCOEFF) [29].

Handerkennung

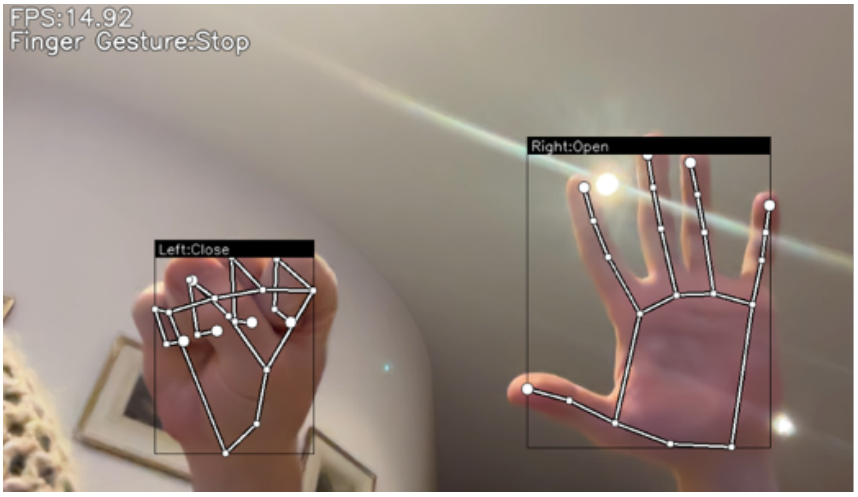


Abbildung 4.2: MP Gestenerkennung.

Gesten können grundsätzlich sehr vielfältig sein; von einer Faust bis zu einem Friedenszeichen. Für die Erkennung solcher Gesten hat Google ein Open-Source-Framework namens MediaPipe (MP) entwickelt. MP hat viele unterschiedliche Anwendungsbereiche, einer davon ist die Gestenerkennung. MP übermittelt aber nicht - wie viele andere KI-Modelle - die einzelnen Pixel als Vektor in ein KI-Modell.

MP führt die Pixel erst in ein Modell ein, um 21 sogenannte *Landmarks* herauszufinden, also Orientierungspunkte. Bei der Hand sind diese beispielsweise die Handwurzel oder Gelenke (siehe die weissen Punkte in Abbildung 4.2) [30]. Die Landmarks werden nun in ein zweites KI-Modell gespiesen. Durch diese Vorgehensweise muss das zweite Modell nicht mit allen einzelnen Pixeln arbeiten, sondern bekommt als Input die 21 Orientierungspunkte. Obwohl das Modell zur Handmerkmalserkennung für sich genommen zeitaufwändig ist, wird sein Suchbereich durch die Handflächenerkennung so stark eingegrenzt, dass die Gesamtzeit für eine Erkennung deutlich gesenkt werden kann. Die Landmark Detection wurde von Google mit circa 30'000 echten und gerenderten Bildern von Händen, jeweils vor unterschiedlichen Hintergründen, trainiert. Das Modell selbst enthält zwei Modelle: eines zum Finden von Handflächen und ein weiteres für die Landmarks. Beide dieser Modelle zusammen entsprechen in der Abbildung 4.3 dem ersten Pfeil. Es werden unterschiedliche Schritte zur Gestenstenerkennung ausgeführt: Links das Bild (bestehend aus Pixeln), in der Mitte die 21 Orientierungspunkte oder Landmarks, rechts die Gestenerkennung. Dabei wird die Händigkeit, sowie Geste abgeleitet, eine Bounding-Box um die Hand gezogen und der einzelnen Landmarks als weisse Punkte dargestellt. Die beiden Pfeile repräsentieren die KI-Modelle. Der zweite Pfeil, also das Modell zur Klassifizierung von Gesten anhand den Landmarks wird nicht von Google zur Verfügung gestellt.

Bei Bewegtbildern wird das Modell zur Handflächenerkennung zeitweise ausgesetzt und die Handmerkmalserkennung sucht am letzten, durch die Handflächenerkennung definierten, Ort [31]. Falls die Handmerkmalserkennung keine Hand erkennt, weil diese sich beispielsweise nach rechts bewegt hat, wird die Handflächenerkennung wieder eingesetzt.

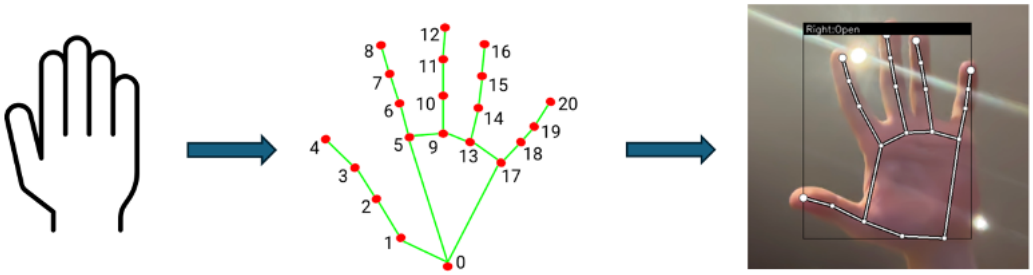


Abbildung 4.3: Unterschiedliche Schritte der Gestenerkennung.

Dadurch kann die Nutzung der Handflächenerkennung ebenfalls minimiert werden. Deshalb läuft aufwändige Bilderkennungs- und -Klassifizierungsaufgabe auch auf kleinen Computern, wie dem RPI, mit hoher Geschwindigkeit.

Um an die Bilder zu kommen, können diese entweder als konventionelle Bilddateiformate direkt importiert werden oder als Numpy-Arrays mittels OpenCV geladen werden – um Webcams nutzen zu können [32]. Für die Bildverarbeitung muss man folgendes tun:

```
import mediapipe as mp

# Konventionelle Bilddatei laden.
mp_image = mp.Image.create_from_file('/path/to/image')

# Bild mittels Numpy Array laden.
mp_image = mp.Image(image_format=mp.ImageFormat.SRGB, data=
    numpy_image)
```

Für Videos und Livestreams geht es ähnlich:

```
import mediapipe as mp

# VideoCapture von OpenCV fuer Aufnahme benutzen.
# Ueber die einzelnen Bilder iterieren.

# Bilder zu MediaPipe Images konvertieren.
mp_image = mp.Image(image_format=mp.ImageFormat.SRGB, data=
    numpy_frame_from_opencv)
```

Anschliessend wird Inferenz ausgelöst – also das neurale Modell ausgeführt. Die Funktion macht Vorverarbeitung der Eingabe, die Erkennung von Händen im Bild und anschliessende Landmarkenerkennung [5]:

```
# Hand Landmark detection fuer ein einzelnes bild
# Dieses muss im Video Modus aufgenommen sein.
hand_landmarker_result = landmarker.detect_for_video(mp_image,
    frame_timestamp_ms)
```

[5] MP braucht zudem noch Zeitstempel.

Die Ausgabe besteht aus einem *Ergebnisobjekt*, mit Landmarks in Bildkoordinaten, Landmarks in Weltkoordinaten und die Händigkeit – also linke oder rechte Hand.

```
HandLandmarkerResult:
  Handedness:
    Categories #0:
      index         : 0
      score         : 0.98396
      categoryName  : Left
  Landmarks:
    Landmark #0:
      x             : 0.638852
      y             : 0.671197
      z             : -3.41E-7
    Landmark #1:
      x             : 0.634599
      y             : 0.536441
      z             : -0.06984
    ... (21 landmarks for a hand)
  WorldLandmarks:
    Landmark #0:
      x             : 0.067485
      y             : 0.031084
      z             : 0.055223
    Landmark #1:
      x             : 0.063209
      y             : -0.00382
      z             : 0.020920
    ... (21 world landmarks for a hand)
```

Im Objekt zu sehen ist die *Handedness*, wobei Index für die Hand steht (MP kann mehrere Hände gleichzeitig erkennen), ausserdem gibt es einen Score (der eine Art Wahrscheinlichkeit oder Sicherheit des Modells darstellt und ob es die rechte oder linke Hand ist). Danach werden die Landmarks angegeben. Wirklich interessant ist hier, dass die Koordinaten dreidimensional sind – obwohl das Eingabebild zweidimensional ist. Hierbei sind x- und y- Werte normalisierte Bildkoordinaten. Diese liegen also zwischen 1 und 0. Symmetrisch für y:

$$x_{NORMALISIERT} = x_{PIXEL-POSITION} / Bildbreite$$

Für die z-Koordinate wird eine x-/y-Ebene gewählt, die durch die Handwurzel geht – also Landmark #0. Deshalb ist dort auch die Grösse sehr klein. Alle anderen z-Koordinaten sind ungefähr gleich skaliert wie die x-/y-Koordinaten und beschreibt die Markierungstiefe. Je tiefer der x-Wert ist, desto näher ist ein Punkt an der Kamera. Die World Landmarks geben die Positionsdaten in Weltkoordinaten an [33]. Das heisst in Metern. Der Koordinatenursprung liegt im geometrischen Mittelpunkt der Hand, also etwa in der Mitte der Handfläche. Es ist dabei unklar, ob die Achsen gleich gewählt wurden wie oben oder im Bezug zum Raum. Um diese Landmarks weiter zu verarbeiten, kann man die Daten entweder in ein zweites Modell transferieren (wie in Abbildung 4.3 gezeigt) oder man kann diese auch direkt nutzen, um einfache Gesten zu erkennen.

1. Zur Erkennung einer Geste wird festgestellt, ob die Distanz zwischen der Spitze des Zeigefingers und der Daumenspitze einen definierten Schwellenwert unterschreitet. Wird diese Bedingung erfüllt, da sich die beiden Finger berühren, wird dies als Steuergeste interpretiert.

2. Ansonsten gibt man die Orientierungspunkte in ein zweites KI-Modell, welches davon die Gesten ableitet. Dieses zweite Modell wird meistens für spezifische Anwendungen trainiert. Als Input nimmt es die eben besprochenen Daten und kann, weil die Datenmenge nicht gross ist, sie normalisiert wurden und immer denselben Ursprung haben, vergleichsweise effizient Gesten bestimmen.

Die Landmarkerkennung kann über die Website von MP genutzt werden [34].

Implikation

Um also ein trainiertes Modell erhalten, wurde (zusätzlich zu MP) auf ein Github ein Repository (kurz Repo) zugegriffen [35]. In der Regel läuft die Handlandmarkerkennung auf dem RPI konstant mit 10 Bildern pro Sekunde (fps), was für diese Anwendung ausreicht. Als nächstes wird sich der Hardware, also der KI-Drohne, gewidmet.

Kapitel 5

Konstruktion und Basis-Hardware der KI-Drohne

5.1 Einführung

Aus Komplexitäts- und Sicherheitsgründen wurde der Drohnenbau im Vorfeld geplant, danach zusammengebaut und unter möglichst sicheren Bedingungen getestet.

5.2 Methodik

Nun werden die Komponenten und ihre Funktionen und Software aufgeführt (siehe Tabelle 5.1). Die Funktion der Komponenten, die noch nicht bereits bei der kleinen Drohne besprochen wurden, werden daraufhin thematisiert.

KOMPONENTE	FUNKTION	SOFT-/FIRMWARE
Pixhawk 6c	FC	ArduPilot
Raspberry Pi 5	CC	Raspberry Pi OS (Linux)
MacBook Air	Laptop	MacOS X und QGroundControl
Holybro Dev-Kit X500V2	Drohnen Kit	
3700 mAh, 14.8 V, 45C, 4S, XT60 Stecker	Batterie	
RadioMaster TX16S MKII	RC-Fernbedienung	
RadioMaster R81 Receiver	Receiver	
Picamera2 – später USB Kamera von Logitech [6]	Kamera	

Tabelle 5.1: Hardware- und Software-Komponenten nebeneinander.

[6] Vorerst wurde die native Kamera des RPI benutzt, jedoch gab es Treiber-Probleme, weswegen später auf eine USB-Webcam gewechselt wurde.

Drohnen-Kit

Aufgrund der Komplexität des Gesamtaufbaus wurde anstelle einer Konstruktion aus Einzelteilen auf ein Drohnen-Kit zurückgegriffen, das alle notwendigen Komponenten (wie ESCs, Motoren und den Rahmen) umfasste [36]. Dennoch gibt es genug Möglichkeiten, die Drohne zu modifizieren und anzupassen. Ausserdem gibt es in der benötigten Grösse (die Kamera und CC heben kann) ausreichend Auswahl und der Community-Support für dieses Kit und auch den FC (bzw. die Firmware und Software davon) ist gross.

FC-Firmware

Bei der ersten Drohne konnte der FC selbst programmiert werden. Hier wird stattdessen eine Open-Source-Software namens ArduPilot (kurz AP) verwendet [37]. Es gäbe auch PX4, jedoch ist sowohl die Dokumentation als auch der Community Support bei AP besser. Beim FC kamen vor allem Geräte von Holybro in Frage [38]. Diese bieten Professionalität, gute Dokumentation, Modularität und einen erschwinglichen Preis. Entsprechend wurde Drohnenkit (mit FC) von Holybro angeschaft [39].

5.3 Übersicht

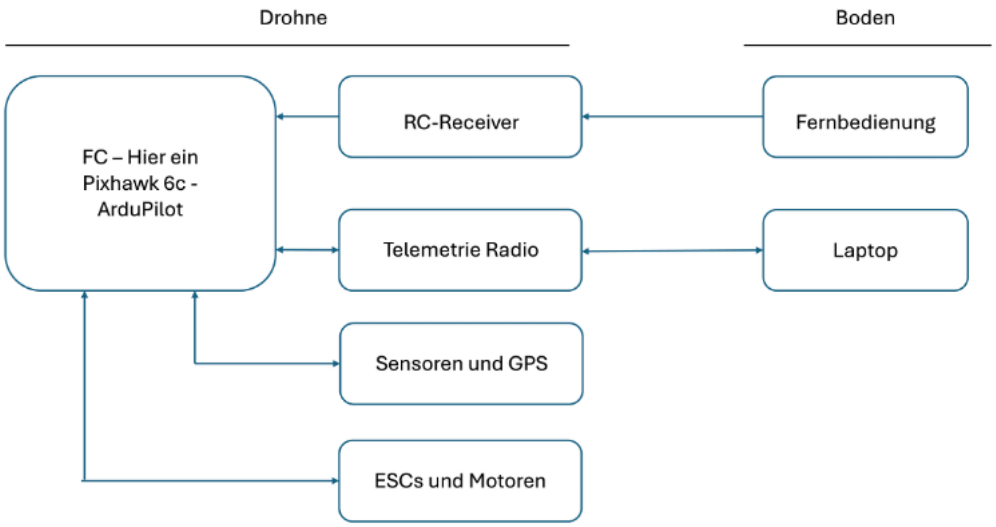


Abbildung 5.1: Übersicht über den Informationsfluss der Drohne ohne CC.

Zuerst wird die Drohne wie in Abbildung 5.1 beschrieben funktionieren. Interessant zu sehen ist vor allem, dass es zwei Verbindungen zum Boden gibt. Die Verbindung zur Fernbedienung gibt dabei nur einfache Steuersignale und die Drohne wird manuell über diese Verbindung geflogen. Die Verbindung zum Laptop ist eine deutlich kompliziertere

Datenverbindung (dazu später mehr). Durch diese können Daten der Drohne noch während dem Flug ausgelesen werden.

5.4 Bau

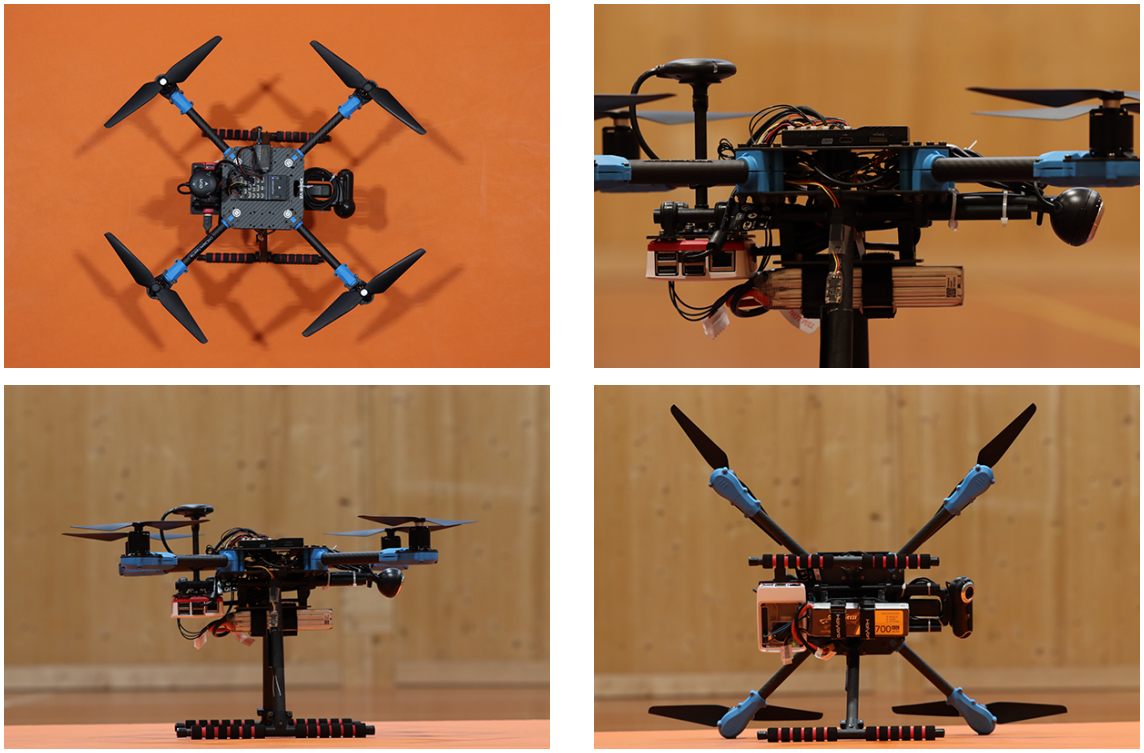


Abbildung 5.2: Drohne aus verschiedenen Perspektiven.

In Abbildung 5.2 wird die Drohne aus verschiedenen Perspektiven gezeigt: Links oben eine Aufsicht (rechts die Kamera, auf der Platte der FC und das Telemetrie Radio, weiter Links das GPS), rechts oben eine detaillierte Seitenansicht (in dem Zwischenraum der Platten ist die Stromverteilung, Kabelmanagement und ein SBUS zu PWM Konverter, unten ist der SBUS-Receiver zu sehen, dahinter ist der Befestigungsort des Akkus, darüber der Step-Down Konverter, rechts die Kamera, oben links das GPS und die Box links-unten ist der RPI), links unten eine Seitenansicht (an dem vorderen Bein ist die RC-Antenne angemacht, in den blauen Gehäusen rechts und links befinden sich die ESCs, die schwarzen Boxen darüber sind die Motoren), rechts unten eine Untersicht (links der RPI mit den drei UART Verbindungskabel, einem USB-C Anschluss für Strom und dem USB-A Anschluss für die Kamera, rechts die Webcam, in der Mitte die Befestigung für den Akku und Kabelbinder um die Hundeleine zu befestigen).

Beim Zusammenbauen traten kleine Probleme auf. Viele Bauteile, darunter: FC, GPS und das Telemetrie Radio wurden angeklebt, um diese vor Vibrationen zu schützen (siehe Abbildung 5.2. Die Drohne hat eine Befestigungsplatte, wo der RPI mit vier Schrauben befestigt ist. Bei Tests wurde der RPI abgeklebt, sodass kein Dreck auf das Board kam – es

gäbe eine Abdeckung mit Lüfter, die aber wegen der UART-Kabel nicht verwendet werden kann. Die Schrauben des RPI wurden mit Sicherungsmuttern und durch Vorspannung vor Vibrationen geschützt. Auf der Vorderseite hat es kleine herausragende Stäbchen, an denen später die Kamera befestigt wird. Die Kamera wurden mit Kabelbindern befestigt. Die ESCs sind in unter den blauen Abdeckungen am Ende jedes Arms. Für das GPS hat es einen Mast, damit dieses exponiert ist – denn das GPS hat einen Deaktivierungsknopf, weshalb man es gut erreichen können muss. Der FC wurde auf die obere Platte geklebt, da man bei diesem Kabel ein- und ausstecken können muss. Der Akku wird mit zwei Bändern festgemacht und ist vor Vibrationen und Unfällen geschützt. An eben diesen Bändern wurde auch eine Hundeleine mit Kabelbindern befestigt. Für die notwendige Stabilität – insbesondere beim Ziehen an der Leine – muss der Aufhängungspunkt exakt unterhalb des Schwerpunkts und des Auftriebsschwerpunkts liegen. Zwingend erforderlich für die Flugstabilität ist zudem, dass der Schwerpunkt im oder leicht unterhalb des geometrischen Mittelpunkts der Motorachsen positioniert ist.

Einrichten

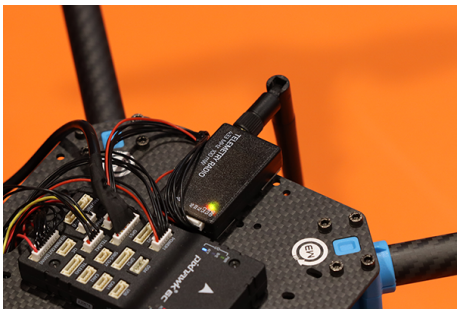


Abbildung 5.3: Telemetrieradio.



Abbildung 5.4: Kalibrierung der Drohne.

Für die Software wurde der FC (via Datenkabel) an den Laptop angeschlossen und die neuste Version von AP darauf geladen. Anschliessend wurden alle Systeme den Vorgaben entsprechend kalibriert (siehe Abbildung 5.4). Später kommunizieren die beiden Geräte (FC und Laptop) über ein sogenanntes Telemetrie Radio (siehe Abbildung 5.3), welches mit dem Laptop verbunden ist. Dabei werden an beiden Geräten Radios (mit Empfänger und Sender) angeschlossen, die kontaktlos miteinander kommunizieren [7].

Lauftest

Damit die Drohne nicht ungewollt abhebt, wurde die Drohne (im flugfähigen Zustand) mit Spanngurten an einem Tisch befestigt (siehe Abbildung 5.5). Im Rahmen des Lauftests sollen die Propeller laufen gelassen werden, um zu überprüfen, ob alles trotz Vibrationen zusammenhält, die Software richtig eingestellt ist und die Motoren richtig drehen.[8]

[7] Interessant zu wissen ist, dass das Telemetrie Radio an der Drohne über die gleiche UART-Schnittstelle und das gleiche Protokoll kommuniziert, wie der RPI später.

[8] Dass die Motoren in die falsche Richtung drehen ist ein typisches Problem. Bei dem Test kann man ein solches Fehlverhalten zuvor ausschliessen. Im Feld würde dieses Problem unweigerlich zu einem Absturz führen.



Abbildung 5.5: Befestigte Drohne vor dem Lauftest.

Der FC selbst hat Sicherheitsvorkehrungen. Er durchläuft vor allem den *Prearming-Check* und das *Arming*. Beim Arming wird die Drohne angelassen. Ähnlich wie beim Auto drehen sich danach die Propeller im Leerlauf. Damit man die Drohne armen kann, muss ein sogenannter Prearming-Check absolviert werden. Das ist ein automatisierter Sicherheitscheck, der beispielsweise die GPS-Verbindung und alle weiteren Sicherheitsmechanismen überprüft.

Beim Prearming-Check ist es ungefährlich, die Drohne anzufassen. Wenn die Drohne das Arming durchlaufen hat, sollte man sich nicht mehr in der Nähe der Drohne befinden, weil die Propeller drehen. Das Arming erfolgt aus diesen Gründen ausschliesslich ferngesteuert. Danach lässt sich die Drohne mit den Joysticks der Fernbedienung kontrollieren.

5.5 Rechtliches im Umgang mit Drohen

Beim Einsatz von Drohnen ist es wichtig, sich über die rechtlichen Vorgaben im Umfeld zu erkundigen [40]. Die KI-Drohne wird in der offenen Kategorie mit der Unterkategorie A3 geflogen. Damit lassen sich Drohnen bis zu 25 kg fliegen. Die Drohne dürfte mit ca. 1.5 kg Gewicht auch in der Unterkategorie A2 fliegen, für diese ist jedoch eine schriftliche Prüfung notwendig, welche ziemlich aufwändig gewesen wäre. Weiter zu beachten sind folgende Punkte:

- Absolvieren eines Onlinetrainings inkl. Test.
- 150 m Abstand zu bewohntem Gebiet und zu Unbeteiligten. (In der Schweiz selten zu finden).
- Abstand von Verbotszonen wie Flughäfen, Gefängnissen, Naturschutzgebieten, etc. Hierfür gibt es eine Drohnenkarte vom Bundesamt für Zivilluftfahrt (BAZL)
- Obwohl gesetzliche Vorgaben durch den Flug in geschlossenen Räumen umgangen werden können, ist diese Option aus zeitlichen und logistischen Gründen meist nicht

praktikabel. Hinzu kommt, dass die schlechte GPS-Signalstärke in Gebäuden die Präzision beim autonomen Schweben erheblich beeinträchtigt.

Es macht Sinn, das Risiko möglichst zu minimieren. Deshalb lohnen sich Tests am Boden, um mögliche Fehlerquellen bereits vor dem Flug zu identifizieren und zu beheben.

Erster Testflug



Abbildung 5.6: Drohne beim ersten Testflug.

Die Drohne wurde an einer Hundeleine befestigt, die von mir oder einer involvierten Person gehalten wird (siehe Abbildung 5.6). Dadurch ist ein Wegfliegen oder jegliche unkontrollierbare Situationen ausgeschlossen und ein Unfall mit unbeteiligten Personen sehr unwahrscheinlich. Ausserdem könnte man mit der Hundeleine im schlimmsten Fall auch „landen“. Der erfolgreiche Testflug beendet die Bauphase. Als nächstes wird der CC ins System eingebunden.

Kapitel 6

Systemintegration und Steuerungsdesign

6.1 Einführung

Der FC ist für die KI-Applikation zu schwach und auch aus Firmware-Gründen nicht dafür geeignet. Der Computer, welcher die Bildverarbeitung übernimmt, nennt sich CC [41]. Typischerweise übernimmt der FC die Flugsteuerung und wird mittels Kommandos (über eine UART-Schnittstelle) vom CC, der die KI-Applikation übernimmt, kontrolliert [42]. Das Kommunikationsprotokoll, welches der FC und auch der CC verwenden, um mit der UART-Schnittstelle zu kommunizieren, heisst MAVLink [43].

UART ist eine Schnittstelle, um serielle Datenströme zu übertragen. Dabei werden die Daten über ein einzelnes Kabel gesendet und auf einem anderen Kabel empfangen. Ausserdem braucht es noch eine gemeinsame Erdung. Also insgesamt 3 Kabel für die bidirektionale Kommunikation.

6.2 Verbinden des Companion Computers mit dem Flight Controller

Zunächst geht es darum, die beiden Computer zu verkabeln, danach müssen sie miteinander kommunizieren und zuletzt wird versucht, diese Kontrolle über den FC mithilfe eines Python-Programmes zu automatisieren. Man soll mit Hilfe dieses Skripts die Drohne steuern können.

Verkabeln

Für die UART Verbindung hat der RPI Pins, an welche man Kabel anschliessen kann. Die entsprechenden drei Pins werden dabei mit einer UART-Schnittstelle des FC (TELEM 2 Port) verbunden [9]. Wichtig ist hier, dass man Bluetooth (im /boot/config.txt file) deaktivieren muss, die ansonsten den UART-Port blockieren würden. Der TELEM 2 Port am FC ist ohne weitere Konfiguration zugänglich.

[9] Das ist die gleiche Schnittstelle, die auch das Telemetrie-Radio verwendet. Der FC hat davon zwei.

Kommunikation

Um die Kommunikation zu überprüfen, wurde MavSDK verwendet. Das erlaubt einen Informationsaustausch mittels CLI (Command Line Interface) des CCs [10][44, 45].

```
laeon@raspberrypi:~ $ sudo mavproxy.py --master=/dev/ttyAMA0
Connect /dev/ttyAMA0 source_system=255
Log Directory:
Telemetry log: mav.tlog
Waiting for heartbeat from /dev/ttyAMA0
MAV> Detected vehicle 1:1 on link 0
online system 1
STABILIZE> Mode STABILIZE
AP: ArduCopter V4.6.1 (57591cb8)
AP: ChibiOS: 88b84600
AP: Pixhawk6C 0044004A 33335116 33313637
AP: IOMCU: 410 2003 411FC231
AP: RCOut: PWM:1-16
AP: IMU0: fast sampling 2.0kHz/2.0kHz
AP: Frame: QUAD/X
AP: PreArm: RC not found
AP: PreArm: Hardware safety switch
Received 1045 parameters (ftp)
Saved 1045 parameters to mav.parm
fence present
Flight battery 100 percent
```

Abbildung 6.1: CLI des RPI bei Verbindung mit dem FC mittels MavSDK.

Die ersten beiden Kommandos öffnen die Verbindung (siehe Abbildung 6.1). Weiter unten sieht man den FC, der verschiedene Parameter und Informationen sendet (beginnend mit AP), danach Statistiken und ganz unten sieht man die erste wirkliche Ausgabe, wobei weitere folgen würden. Man kann diese Shell auch benutzen, um beispielsweise den Modus des FCs zu ändern.

[10]Nebenbei bemerkt müssen für die Funktion von MavSDK auch sehr viele sogenannte Dependencies (Abhängigkeiten) installiert werden.

6.3 Schreiben von Python-Skripten

Einführung

Bei der Entwicklung von Drohnensoftware sind detaillierte Sicherheitsüberlegungen unerlässlich, um eine Gefährdung zu vermeiden. Die Fehlerfreiheit in sicherheitsrelevanten Bereichen und ein zuverlässiger Umgang mit unerwarteten Situationen sind dabei zwingend.

Gefahren

Wird der Drohne ein Landebefehl gegeben, ist etwas Zeit erforderlich, bis sie gelandet ist. Wenn die Drohne durch den Befehl *disarmed* die Motoren abstellt, ohne dass zuvor ihre Bodenberührung überprüft wurde, stürzt sie ab. Daher ist eine Überprüfung der Bodenberührung vor dem Entschärfen zwingend erforderlich. Solche Überlegungen muss man sich bei jedem Befehl machen. Denn jeder Befehl an die Drohne kann immer in einem Error oder reaktionslos enden.

Der *Modus* ist bei der Drohne eine Einstellung, die bestimmt, wie und durch was die Drohne gesteuert wird. Beispielsweise gibt es den 'Guided' Modus, in welchem die Drohne die CC Kommandos annimmt, aber auch den 'Pos-Hold' Modus (Position-Hold), in welcher sich die Drohne manuell steuern lässt, aber mit GPS gegen Umwelteinflüsse korrigiert wird. Auch beim manuellen Wechseln in Pos-Hold, passiert ohne weiteres im Programm nichts, denn das Programm versucht noch immer Befehle an den FC zu geben, obwohl sie dieser nicht mehr annimmt. Ganz gefährlich wird es, wenn man im manuellen Modus die Drohne fliegt, das Python-Programm den Modus ändert und ohne Vorwarnung die Kontrollautorität übernimmt. An einem Beispielskript wird die Thematik erläutert.

Beispielskript

```
1 from dronekit import connect, VehicleMode
2 import time
3 import sys
```

Zuerst werden die notwendigen Bibliotheken importiert. Danach geht das Programm zum Hauptprogramm, mit dem die Drohne verbunden wird. Ausserdem wird, für die Ansteuerung durch DroneKit, ein Objekt, welches der verbundenen Drohne entspricht, namens *vehicle* geschaffen.

```
1 vehicle = connect('/dev/ttyAMA0', wait_ready=True, baud=57600)
```

Das Programm versucht darauf, unterschiedliche Funktionen auszuführen. Wenn dabei ein Error entsteht, behandelt es ihn mit Verbindungskappung.[11]

[11] Interessant zu sehen ist, dass vehicle (welches im aktuellen Namespace existiert) in die einzelnen Funktionen gegeben wird, weil es kein globales Objekt ist.

```
try:
    arm_and_takeoff(vehicle, 20)
    time.sleep(200)
    land(vehicle)
    disarm(vehicle)
except Exception as e:
    print(f"An error occurred: {e}")
finally:
    try:
        vehicle.close()
    except:
        pass
```

Bevor die einzelnen Funktionen angeschaut werden, muss eine sicherheitsrelevante Funktion besprochen werden. Nämlich die *check_mode* Funktion. Sollte während dem Flug manuell eingegriffen werden, muss der Modus von Guided auf Pos-Hold geändert werden. Damit das Programm dann keine Befehle mehr sendet und den Modus nicht ändert, soll vor allen Flugmanövern überprüft werden, dass die Drohne im Guided-Modus ist. Je nach Situation soll die Drohne (bzw. das Python-Skript) entweder warten, bis der Modus auf Guided geändert wurde oder das vehicle (bzw. dessen Verbindung) schliessen und das Skript beenden.

```
def check_mode(vehicle, exit_if_not_guided=False):
    #check if the mode is set to guided. if nothing specified, wait
        until it changed to guided, else end the program
    while(vehicle.mode.name.upper() != "GUIDED"):
        if(exit_if_not_guided == True):
            print(f"[SAFETY] Vehicle mode changed to {vehicle.mode.
                name}! Exiting program for safety.")
            try:
                vehicle.close()
                print("Connection closed.")
            except Exception as e:
                print(f"Error closing vehicle: {e}")
            sys.exit("[ABORT] Script stopped because vehicle left
                GUIDED mode.")
        else:
            print("Set vehicle mode to guided!")
            time.sleep(1)
    return
```

Man sieht wie in Zeile 1 vehicle importiert wird und das Programm wartet, bis der Modus Guided ist, wenn aber als zweites Argument *True* in die Funktion gegeben wird, deaktiviert sich das Python-Skript bei einer Modusänderung automatisch. Nun zu einer der Flugfunktionen, der arm_and_takeoff Funktion:

```
def arm_and_takeoff(vehicle, aTargetAltitude, timeout=60):
    # Arms vehicle and fly to aTargetAltitude.

    print("Basic pre-arm checks")
    # Don't try to arm until autopilot is ready

    check_mode(vehicle)

    while not vehicle.is_armable:
        print(" Waiting for vehicle to initialise...")
        time.sleep(1)

    # Copter should arm in GUIDED mode
    print("Arming motors")
    vehicle.armed    = True

    start = time.time()
    while not vehicle.armed:
        if time.time() - start > timeout:
            raise TimeoutError("Arming timed out")
        print(" Waiting for arming...")
        time.sleep(1)

    print("Taking off!")
    vehicle.simple_takeoff(aTargetAltitude) # Take off to target
        altitude

    start = time.time()
    while True:
        check_mode(vehicle, True)
        alt = vehicle.location.global_relative_frame.alt
        print(" Altitude:", alt)
        if alt is not None and alt >= aTargetAltitude * 0.95:
            print("Reached target altitude")
            break
        if time.time() - start > timeout:
            raise TimeoutError("Takeoff timed out")
        time.sleep(1)
```

Zuerst wird die (eben besprochene) Funktion ausgeführt.

```
check_mode(vehicle)
```

Da die Drohne noch am Boden ist, darf das Programm warten, bis der Modus (manuell) auf Guided gestellt wurde [12]. Dann wird überprüft, ob das Prearming erfolgreich war.

```
while not vehicle.is_armable:
```

[12] Obwohl ein automatisierter Moduswechsel technisch realisierbar wäre, wurde bewusst darauf verzichtet, um unvorhergesehene Kontrollübergaben zu vermeiden. Denn die Verschiebung der Kontrolle vom Piloten auf das Skript birgt erhöhte Sicherheitsrisiken und untergräbt die menschliche Steuerungsautorität. Lediglich für den Landemodus war eine automatische Lösung notwendig.

Es wird versucht, die Drohne zu armen.

```
vehicle.armed    = True
```

Man muss sicherheitshalber aber nochmals überprüfen, ob das Arming erfolgreich war.

```
start = time.time()
while not vehicle.armed:
    if time.time() - start > timeout:
        raise TimeoutError("Arming timed out")
    print(" Waiting for arming...")
    time.sleep(1)
```

Es wird gewartet bis zum *Timeout*, also eine Variable, deren Standartwert in der Funktionsdeklaration bei 60 Sekunden liegt, den man jedoch auch anders setzen könnte, dann kommt der Befehl zum Start:

```
vehicle.simple_takeoff(aTargetAltitude)
```

Wie auch schon beim Armieren, muss auch diese Funktionsausführung überwacht werden.

```
start = time.time()
while True:
    check_mode(vehicle, True)
    alt = vehicle.location.global_relative_frame.alt
    print(" Altitude:", alt)
    if alt is not None and alt >= aTargetAltitude * 0.95:
        print("Reached target altitude")
        break
    if time.time() - start > timeout:
        raise TimeoutError("Takeoff timed out")
    time.sleep(1)
```

Die Höhe wird überprüft und sobald diese der Zielhöhe entspricht, hat die Funktion ihren Zweck erfüllt. Weitere Funktionen sind ähnlich aufgebaut. Interessant ist dabei, dass bei der Funktion zum Landen ein Moduswechsel gemacht werden muss, dieser ersetzt konzeptuell den Startbefehl von oben. Zusätzlich muss man gerade auch beim sogenannten *Disarming* (also dem Gegenteil des Armings bzw. zu Deutsch entschärfen) überprüfen, dass die Drohne nicht mehr fliegt. Zuletzt wird die Verbindung beendet:

```
vehicle.close()
```

6.4 Flug kontrolliert durch den CC

Die Drohne wurde mit dem Programm von Kapitel 6.3 getestet. Dabei ist sie auf eine Höhe von 3 Metern gestiegen, hat dort die Höhe gehalten und ist anschliessend wieder gelandet. Das Programm funktionierte dabei wie geplant [13].

[13] Davor wurden noch weitere Tests zur manuellen Übernahme gemacht. Falls das Skript nicht wie gewollt funktionieren würde, muss man die Drohne manuell übernehmen können.

6.5 Gestensteuerung auf dem RPI

Aus Zeitgründen wurde auf ein Beispielprogramm von Google bzw. MP selbst umdisponiert, welches bloss die Handorientierungspunkteerkennung hat [46]. Auf dieses Beispielprogramm wurde aufgebaut.

Grundsätzliches

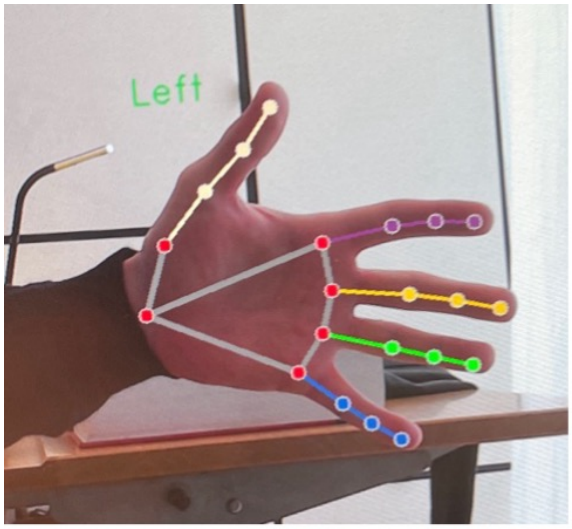


Abbildung 6.2: Mediapipe auf dem RPI.

Von der im Kapitel 4.3 besprochenen Gestenerkennung handelt es sich um die Handerkennung und Landmarken Detektion (siehe Abbildung 6.2). Grundsätzlich kann man auch von den Daten der Landmarken bereits Gesten ableiten – ohne KI, indem man Werte wie die x-Koordinaten vergleicht. So kann man zum Beispiel herausfinden, ob und wie die Hand gedreht ist. Konzeptuell könnte man die Drohnensteuerung ins gleiche Programm einbetten, jedoch schafft das zwei Probleme:

1. Die Lesbarkeit der Programme würde drastisch sinken, da ganz unterschiedliche Aufgaben im gleichen Code gelöst würden. Dadurch sinkt die Lesbarkeit und steigt das Risiko für fehlerhaften Code.

2. Es gäbe Probleme, weil die verschiedenen Programmteile unterschiedliche Dependenzen haben und in diesem Fall somit inkompatibel wären.

Deshalb ist es sinnvoll, zwei separate Programme zu schreiben. Eines für die Bilderkennung und Verarbeitung der Daten, ein Zweites für die Steuerung der Drohnen anhand der Daten des ersten Programmes.

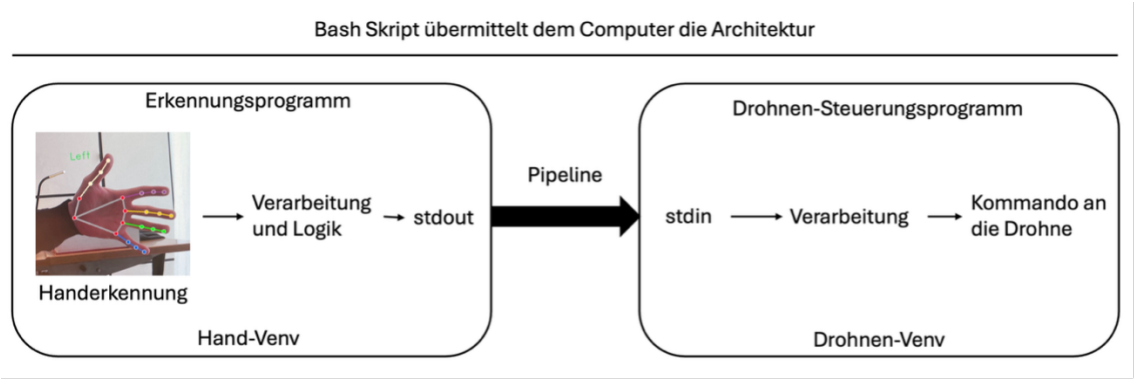


Abbildung 6.3: Logik des Computerprogramms im RPI.

Venv

Da dieses Programm für die Orientierungspunkteerkennung und das Programm der Drohnensteuerung wiederrum unterschiedliche (miteinander nicht kompatible) Pakete benötigen, hat jede Applikation ein jeweiliges Venv (Virtual Environment). Ein Venv ist eine Umgebung, in welche man Pakete installiert, um sie nicht systemweit zu installieren. Das Venv wird während oder vor dem Ausführen eines Programmes aktiviert. Es wurde ein kurzes Bash Skript nötig, um die beiden Programme mit dem jeweiligen Venv laufen zu lassen und um eine Pipeline zu haben, die den Standardoutput des Handerkennungsprogrammes in den Standardinput des Drohnensteuerungsprogrammes gibt (siehe Abbildung 6.3). Darin entsprechen die Boxen der Einteilung in zwei Programme. Das Handerkennungsprogramm (hier links) verarbeitet das Bild der Webcam zu einer jeweiligen Bewegungsentscheidung. Also beispielsweise, ob die Drohne nach rechts oder links drehen soll. Dies wird als Standartausgabe mittels einer Pipeline in die Standarteingabe des Steuerungsprogrammes gegeben. Dort wird nach Bedarf die aktuelle Sendung aus dem Buffer ausgelesen – der Buffer wird ebenfalls geleert. Anschliessend wird das Datenpaket aufgeteilt und für die Drohnensteuerung zur Verfügung gestellt.

Konkrete Applikation

Die Drohne soll sich zur Zeigerichtung des Daumens hindrehen. Dafür muss sie wissen, ob sie nach rechts, links oder nicht drehen soll. Das findet sie heraus, indem überprüft wird, ob der Daumen die am stärksten rechte/linke landmark ist. Dieses Resultat kann man nun weiterschicken an das andere Programm:

```
min_other_x = min(all_other_x_coords)
max_other_x = max(all_other_x_coords)
if finger_tip_x < min_other_x:
    tip_position_output = "Left"
elif finger_tip_x > max_other_x:
    tip_position_output = "Right"
else:
    tip_position_output = "None"
print(f"Hand␣{idx}:␣({xmin},␣{ymin}),␣({xmax},␣{ymax}),␣{
   tip_position_output}", flush=True)
```

Thematisch gehören Entscheidungen in das Programm der Objekterkennung. Nun gibt also das Programm der Objekterkennung nur noch Entscheidungen (wie: Rechts) an das Drohnenprogramm weiter. Das Drohnenprogramm liest anschliessend den Buffer des Standartinputs aus [14]:

```
def hole_letzte_zeile():
    letzte_zeile = None
    try:
        while True:
            i, _, _ = select.select([sys.stdin], [], [], 0.0) # (
               Timeout 0.0)
            if i:
                zeile = sys.stdin.readline()
                if zeile:
                    letzte_zeile = zeile.strip()
                else:
                    break # EOF erreicht, beende die Schleife
            else:
                break # select meldet: Puffer ist leer. Breche das
                   Lesen ab.
    except ValueError:
        # sys.stdin ist geschlossen
        pass
    except Exception as e:
        # Andere Fehler beim Lesen
        print(f"Fehler␣beim␣Lesen:␣{e}", file=sys.stderr)
    return letzte_zeile
```

Das zweite Programm nimmt also die aktuellste Bestimmung, sodass die Latenz reduziert wird. Die *return*-Werte gehen später in die Steuerung ein. Man muss sich hier überlegen, was bei einem leeren Buffer geschieht (falls das Erkennungsprogramm langsamer laufen sollte, wie das Drohnenprogramm. Umgekehrt bedeutet ein voller Buffer auch Verzögerungen beim Auslesen. Das führt wiederrum zu einer höheren Latenzzeit [15].

[14] Die Koordinaten werden zwar übermittelt, das zweite Programm fängt damit aber nichts an. Sie sind ausschliesslich dazu da, um bei Tests die Position der Hand im Bild zu erkennen.

[15] Zu einem Buffer-Overflow kommt es bei dem Standardinput eines Python-Programmes nicht, da der Speicher dynamisch verwaltet wird (sogenanntes Buffering).

Kapitel 7

Ergebnisse

7.1 Minimalistische Drohne

Eckdaten

Länge (ohne Propeller)	13 cm
Breite (ohne Propeller)	19 cm
Länge (mit Propeller)	18 cm
Breite (mit Propeller)	25 cm
Höhe (mit Antenne)	14 cm
Höhe (ohne Antenne)	7 cm
Gewicht (ohne Akku)	180 g
Gewicht (mit Akku)	260 g
Nutzlast[16]	80 g
Maximale Dauerleistungsaufnahme der Batterie[17]	330 W

Tabelle 7.1: Eckdaten der minimalistischen Drohne.

Flugverhalten: minimalistische Drohne

Beim Flug in einer Sporthalle fing die Drohne in Bodennähe an zu *flattern*. Beobachten liess sich dieser Effekt nur in Bodennähe, beim Flug war die Drohne ruhig. Die Drohne driftete auch immer leicht in eine Richtung, trotz einer präzisen Kalibrierung.

[16] Zur Nutzlast gehört alles, was mehr mitfliegen kann als das Leergewicht. (Also Akkus, Kameras, CC, ...). Beim Transportieren dieser Nutzlast wird etwa 70% der Maximalleistung abgegeben.

[17] Berechnet nach der maximalen Dauerleistungsabgabe von Kapitel 3.2

7.2 Eckdaten: KI-Drohne

Länge (ohne Propeller)	41 cm
Breite (ohne Propeller)	41 cm
Länge (mit Propeller)	61 cm
Breite (mit Propeller)	61 cm
Höhe	31 cm
Gewicht (ohne Akku)	1390 g
Gewicht (mit Akku)	1740 g
Nutzlast	1500 g
Maximale Dauerleistungsaufnahme der Batterie	2700 W
Leistungsaufnahme bei 70% Throttle	700 W
Leistungsaufnahme bei 100% Throttle	1200 W

Tabelle 7.2: Eckdaten der KI-Drohne.

7.3 Generelles Flugverhalten

Damit die Drohne vom RPI gesteuert werden kann, muss sie manuell in den GUIDED-Modus gesetzt werden. Danach hat man selbst keine Kontrolle. Um die Kontrolle wieder zu erhalten, muss der Modus zurückgesetzt werden. Dann kann die Drohne wieder wie gewohnt von Hand geflogen werden.

Das Programm übernimmt generell niemals selbst die Kontrollautorität. Wenn die Drohne am Boden ist, wartet das Programm, bis es im GUIDED Modus ist. Falls während dem Flug der Modus geändert wird, hat die Fernsteuerung direkt die Kontrollautorität und das Programm beendet die aktuelle Aufgabe, kappt die Verbindung zur Drohne und deaktiviert sich.

Windresistenz

Die Drohne lässt sich bei spürbar starken Winden nicht mehr sicher starten, da sie sonst beim Start umkippt. Es muss also auf ein windstilles Umfeld geachtet werden. In der Luft verträgt sie Winde bis zu circa 10 km/h. Jedoch hat die Stabilisation mittels GPS Probleme bei starken Winden.

7.4 Testsoftware

Die Testsoftware umfasst:

1. Handerkennung.
2. Handlandmarkerkennung.
3. Verarbeitung der Handlandmarken zur Unterscheidung in Gesten (mit KI).
4. Verarbeitung der Handlandmarken zu simplen Gesten (ohne KI).
5. Unterschiedliche Venvs.
6. Bash Skripte zum Ausführen komplizierterer Programme.
7. Arming Tests.
8. Flugtests.
9. Flugtests mit Flugmanövern.
10. Einbettung der Gesten in den Flugmanövertest.

Damit verbunden ist die Benutzung einer Kamera, KI und die Kommunikation mit mehreren anderen Computern, darunter der CC.

7.5 Testverfahren

In dieser Arbeit wurden verschiedenste Test-Skripte geschrieben. Eines davon wurde teilweise besprochen. Auch wurden zwei Orte gefunden und genutzt, an denen man Drohnen der Kategorie A3 fliegen darf:

- 46°48′49.4″N 9°13′22.4″E - Aargau
- 47°18′15.7″N 8°21′09.9″E - Graubünden

Ausserdem wurden die Sporthallen der Kantonsschule Freudenberg als Testorte benutzt.

- 47°21′56.2″N 8°31′42.2″E - Zürich

Während der Tests wurde die Drohne jeweils an einer Hundeleine befestigt um auszuschliessen, dass die Drohne die Gegend verlässt. Ausserdem konnte jederzeit mit der Fernbedienung die manuelle Kontrolle übernommen werden. Der Gashebel der Fernbedienung muss beim Flug immer in die neutrale Position gebracht werden, sodass die Drohne bei manueller Übernahme nicht abstürzt. Bei dieser Drohne muss man aufpassen, dass der Untergrund flach und gerade ist, damit die Drohne nicht kippen kann. Die Batteriespannung muss während dem Test ungefähr alle 5 min und sobald der RPI angeschlossen ist, die Drohne aber nicht fliegt, ungefähr alle 20 min gemessen werden.

7.6 Zuverlässigkeit der Handerkennung

Während der gesamten Arbeit wurde nie eine Hand detektiert, wo keine war. Die Landmarken sind sehr genau. Sie weichen zwar oft um ein paar Millimeter von Gelenken ab, waren jedoch fast nie ausserhalb der Hand. Die Hand wird in den unterschiedlichsten Positionen erkannt. Selbst wenn Finger nicht zu sehen sind oder nur ein paar Pixel eines Fingers zu sehen sind, gibt das Modell sehr verlässliche und genaue Daten an. Gesten wurden jeweils auch mit Präzision erkannt, sobald die Hand erkannt wurde. Bei Tests wurden Gesten bis auf 7 m korrekt erkannt. Probleme gab es nur, wenn viele Finger nicht mehr zu sehen sind. Also wenn man beispielsweise den Handrücken zeigt. Wie auch in der Abbildung 7.1 zu sehen.

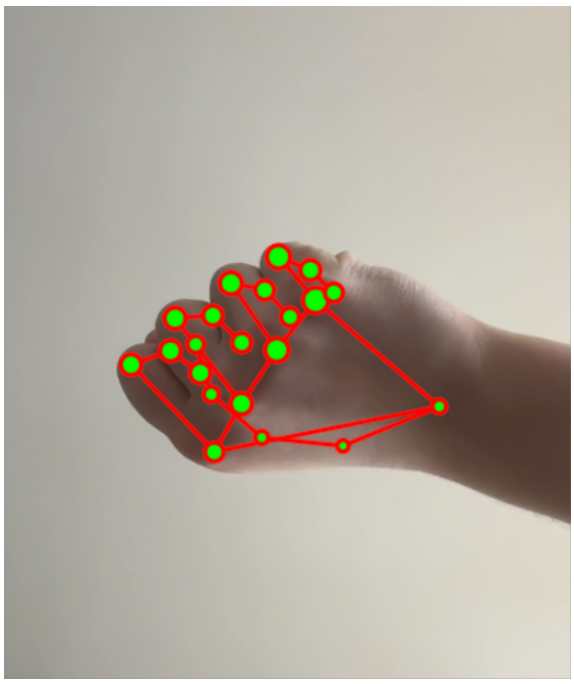


Abbildung 7.1: Grenzen von Mediapipe beim Zeigen eines Handrückens.

Geschwindigkeiten

Sämtliche Mediapipe-Anwendungen liefen auf dem Laptop mit ca. 30 fps. Auf dem RPI haben sie üblicherweise je nach Anwendung zwischen 7 und 10 fps, trotz der eher schwachen Hardware für KI. Diese Bildwiederholrate ist ausreichend, um die Drohne zu steuern [18].

Sonstiges

Die Interaktion mit dem RPI ist derzeit nur über ein CLI möglich (abgesehen von der Kamera). Auf diese Shell kann vom Boden aus mittels SSH-Verbindung zugegriffen werden.

[18] Grundsätzlich gilt bei gleichbleibender Drehgeschwindigkeit der Drohne: Je höher die Bildwiederhol- und verarbeitungsrate ist, desto präziser lässt sich die Drohne steuern.

7.7 Charakterisierung der Verbindung

WLAN

Durch Weglaufen wurde die maximale Distanz vom RPI (WLAN 5) zum iPhone Hotspot (WLAN 6) auf grob 30 m gemessen. Garantieren lässt sich eine genügende Verbindung bis hin zu 20 m.

RC-Verbindung

Messungen der RC-Verbindung sind technisch nicht machbar. Das Signal dürfte allerdings ca. 100 m weit kommen. Falls die Drohne die Verbindung verliert, kommt sie zum Startpunkt zurück und landet.

7.8 Modifikation der Drohne

Das Drohnenkit wurde mit folgender Ausstattung ergänzt:

- RPI.
- Kamera.
- Step-Down Konverter.
- Hundeleine.

Die Robustheit der Montage wurde durch sorgfältige Beobachtung der Anbauteile während aller Testphasen validiert – auf separate Festigkeitsprüfungen wurde verzichtet. Dabei wurden keine besorgniserregenden Ergebnisse erzielt: Es traten keine signifikanten Vibrationen oder Schütteln im Betrieb auf.

Kapitel 8

Diskussion

8.1 Erreichen der Ziele

Gemäss der Fragenstellungen war das Ziel, zu ergründen, wie man eine KI-Applikation bei einer Drohne implementiert. Darüber hinaus sollte der Aufbau der dafür benötigten Plattform, also die Drohne mit den sich darauf befindenden Systemen, für die Arbeit zentral sein. Nachfolgend noch einmal die Fragen mit kurzen Antworten:

1. Wie funktioniert Gestenerkennung grundsätzlich?
   Die Gestenerkennung erfolgt durch Echtzeit-Bildklassifizierung mittels eines KI-Modells. Dabei werden die Landmarks der Hand erfasst, deren Position gemessen und diese als spezifischer Steuerbefehl interpretiert.

2. Wie optimiert man KI für Computer mit wenig Rechenleistung?
   Die KI wird optimiert, indem speziell entwickelte, leichtgewichtige KI-Modelle (wie z. B. in Mediapipe verwendet) zum Einsatz kommen, die minimalen Rechenaufwand erfordern, aber dennoch die notwendige Echtzeit-Bildklassifizierung der visuellen Daten ermöglichen. Dies erlaubt die Verarbeitung auf dem Companion Computer.

3. Wie minimiert man das Unfallisiko beim Entwickeln einer Drohne?
   Das Risiko wird durch Sicherheitsüberlegungen zur Unfallvermeidung minimiert. Dies schließt die Eliminierung sicherheitsrelevanter Bugs und die Implementierung eines robusten Fehler- und Ausnahmemanagements ein. Zudem wird die Kontrollautorität primär beim Piloten belassen, indem automatisierte Moduswechsel weitestgehend vermieden werden.

4. Wie funktioniert die Kommunikation und deren Automation bei den unterschiedlichen Bauteilen?
   Der Companion Computer, auf dem das KI-Skript läuft, generiert via Python-Skript Steuerbefehle. Diese werden an den gesendet, welcher die Befehle in direkte Motorsignale umwandelt und die Flugstabilität regelt.

5. Lässt sich die Drohne somit mit Gesten steuern?
   Die von der Kamera erfassten visuellen Daten werden vom KI-Modell einer Bildklassifizierung unterzogen. Die erkannte Geste (z. B. ein nach rechts zeigender Daumen)

wird interpretiert und automatisch in einen korrespondierenden Flugsteuerbefehl umgesetzt, der die Bewegung der Drohne auslöst.

Wie die Tests belegen, konnten zwei flugtaugliche Drohnen zusammengebaut werden. Ausserdem gelang die Einbindung des CCs in das System. Wie hier die Tests, der besprochene Code und weitere Programme belegen, konnte auch der CC vollständig eingebunden werden und die Drohne über den FC kontrollieren.

Die Funktionsweise einer Gestenerkennung, sowie -steuerung wurde ergründet, genauso wie die Optimierung ebendieser Applikation für Computer mit geringer Rechenleistung thematisiert. Weiter wurde die Softwareentwicklung und Risikoreduktion im Zusammenhang mit Python und sicherheitsrelevanten Funktionen besprochen.

Die Sicherheitsvorkehrungen, sowohl bei der Software wie auch der Hardware und bei Tests, wurden durchdacht und entsprechend nach den Vorstellungen umgesetzt. Trotz Unfällen mit den Drohnen und weiteren Problemen bestand nie ein Sicherheitsrisiko. Die Arbeit befand sich stets in einem legalen Rahmen.

8.2 Probleme, Rückschläge und Gelerntes

Portable SSH-Verbindung

Die Secure Shell erlaubt den Zugriff bzw. die Steuerung eines Computers ohne physische Verbindung. Dabei wird die Verbindung über ein Netzwerk geleitet und ermöglicht ein CLI des RPIs während dem Flug. SSH funktioniert so, dass es über das Netzwerk eine verschlüsselte Verbindung vom Laptop zum RPI eröffnet. Dank WLAN ist die Verbindung sogar drahtlos. Das ist notwendig, weil zu der Drohne keine physische Verbindung besteht. Jedoch funktioniert SSH nur, solange beide Geräte im selben Netzwerk sind. Auch draussen! Für diese Verbindung draussen gibt es grundsätzlich zwei Lösungsansätze:

1. Der RPI wird als Access Point benutzt, um die beiden Geräte direkt zu verbinden. Das ist jedoch kompliziert aufzusetzen. Zudem haben bei dieser direkten Verbindung beide Geräte keine Internetverbindung mehr.

2. Man verbindet beide Geräte mit einem lokalen Netzwerk. Wenn man hier überall hingehen will, kann man einen Hotspot von einem Mobiltelefon verwenden.

Da der erste Ansatz zum Verlust der Internetverbindung auf beiden Geräten geführt hätte, musste eine Alternative gefunden werden. Die Standardlösung, den RPI via Ethernet an ein LAN anzubinden, entfiel aufgrund der geforderten Mobilität der Drohne. Man entschied sich daher für Variante 2: die Nutzung eines Hotspots.

Tiefenentladener Akku

Um den RPI beim Flug mit Strom zu versorgen, wurde er mit einem Abwärtswandler (englisch: Step-Down Konverter) an die Batterie angeschlossen. Der Konverter ist notwendig, weil die Batterie der Drohne nicht zuverlässig die gleiche Spannung hat und zusätzlich dazu die Spannung der Batterie für den RPI zu hoch wäre. Es wäre auch möglich gewesen, ihn mit einer separaten Batterie (wie einer Powerbank) zu betreiben, das wäre jedoch



Abbildung 8.1: Aufladen eines tiefenentladenen Akkus.

komplexer und weniger elegant. Man kann den RPI nicht über den 5 V Pin des FCs versorgen, denn auch wenn die Spannung ausreichen würde, braucht der RPI zu viel Strom. Die 5 V Pins wurden z. B. für GPS-Module konzipiert, die deutlich weniger Strom brauchen und sind nicht für deinen RPI geeignet.

Während dem Testen, hatte der RPI plötzlich keinen Strom mehr. Also wurde das Batterieladegerät für eine Messung der Zellspannungen des Akkus angeschlossen. Nachdem dies nichts anzeigte, wurden alle Zellen manuell mit einem Voltmeter gemessen. Die vier Zellen waren bei Spannungen zwischen 2 Volt (V) und 2.5 V. Nur zum Vergleich: Eine LiPo-Zelle sollte man nicht über 4.2 V laden, beim Lagern sollte sie ca. 3.8 V haben und niemals unter 3.5 V gelangen, da sie sonst permanent Schaden nehmen kann. Denn bei geringeren Spannungen verändert sich im Inneren die chemische Struktur. Üblicherweise kann man Akkus unter 2.5 V nicht mehr retten. Später wurden alle Zellen mit Spannungen über 3 V gemessen und das Ladegerät konnte diese wieder aufladen.

Auch beim Aufladen eines tiefenentladenen Akkus muss man sehr vorsichtig sein [19]. Hier wurde der Akku in einem Feuerfesten Box, die für LiPos gebaut wurde, in einem Kamin mit Glas (von grosser Materialstärke) und ohne brennbares Material in der Nähe geladen (siehe Abbildung 8.1). Ausserdem wurden, erst im Abstand von fünf Minuten, später alle zehn Minuten, Spannung, Temperatur und Deformation manuell gemessen. Ausserdem wurde mit 0.2 A aufgeladen (also einem sehr geringen Strom).

Erster Crash

Um nun die Kommunikation und Flugsteuerung durch den RPI zu testen, sollte die Drohne damit geflogen werden. Zuerst wurde der CC ausgesteckt und die Drohne manuell geflogen, um die Balance insbesondere wegen dem (neuen) Gewicht zu überprüfen. Anschliessend wurde (nach vorhergehenden Lauftests) nochmals überprüft, dass sich an der Fernbedienung der Modus umstellen lässt, um die Drohne manuell zu übernehmen. Dazu wurde diese durch den RPI gearmt. Daraufhin wurde per Fernbedienung der Modus zu Pos-Hold

[19] Es ist dringend davon abzuraten, tiefenentladene Akkus aufzuladen. Im Vorfeld hierzu fand eine gründliche Recherche statt.

gewechselt. Die Drohne versuchte also ihre Position zu halten, obwohl sie noch am Boden war. Vermutlich aufgrund von der Ungenauigkeit des GPS gab die Drohne etwas Schub, um die Höhe zu halten. Durch den zusätzlichen Wind an diesem Tag kippte die Drohne schliesslich um. Dadurch wurden zwei Propeller irreparabel beschädigt.

Zweiter Crash

Bei einem weiteren Test in einer Sporthalle wurde trotz schlechter GPS-Verbindung geflogen. Der Flug selbst lief gut, allerdings kippte auch hier die Drohne am Boden um (wobei zwei Propeller zerstört wurden). Das Problem war offensichtlich nicht der Wind, sondern der Versuch, die Position am Boden zu halten. Die Drohne versuchte also, ihre Position gegenüber dem GPS-Signal beizubehalten (welches sich bewegte durch Ungenauigkeit). Da die Drohne aber schon den Boden berührte, kippte sie dabei um. Die Rotoren stellten sich dabei nicht automatisch aus. Für zukünftige Flüge musste also ein Not-Aus Schalter der Propeller aufgesetzt werden, der auch während dem Flug geht.

Gestensteuerung auf dem RPI

Um die Gestensteuerung auf dem RPI laufen zu lassen, wurde das Repo, welches in Kapitel 4.3 für die Gestensteuerung benötigt wurde, auf den RPI geladen. Entsprechend wurden auch die sogenannten Python-Pakete heruntergeladen, wobei es jedoch Versionskonflikte gab. Oft sind Python-Pakete (wie z.B. Opencv) von anderen Paketen abhängig (hier z.B. Numpy). Diese haben wiederrum ihre eigenen Abhängigkeiten (Dependencies). Da man mehrere Pakete braucht, benötigen diese wiederrum eine Grosszahl an Dependencies. Dabei gab es viele Versionskonflikte, da zum Beispiel OpenCV eine neue Version von Numpy brauchte, wobei Tensorflow (eine Dependency von MP) wiederrum eine ältere Version von Numpy benötigte. Deshalb liessen sich die meisten Pakete nicht installieren.

Aufgrund unterschiedlicher Systeme kann das Venv von Mac (wo das Repo funktioniert hat) nicht einfach auf den RPI kopiert werden.[20] Also wurden aus dem Venv auf dem Mac alle Pakete ausgelesen, auf dem RPI automatisiert und in ein natives Venv installiert. Aber selbst da gab es Probleme, weil bei einem Paket die entsprechende Version nicht verfügbar war.

Das Problem wurde im Endeffekt nicht mehr gelöst, es wurde ein anderes Programm verwendet. Siehe Kapitel 6.5.

Kamera

OpenCV funktioniert mit der Pi Camera 2 nicht. Denn diese ist für den RPI gemacht, benutzt eigene Bibliotheken. OpenCV kann weder auf die Bibliotheken noch auf die Kamera zugreifen [21]. Also wurde kurzerhand eine USB-Webcam angeschlossen, was funktionierte, weil OpenCV üblicherweise auf USB-Zubehör zugreifen kann [47].

[20] Ausserdem lief es auch im Mac nicht in einem Docker-Container. Aber um es nun in einem Docker-Container laufen zu lassen, hätte man identische Probleme.

[21] Man müsste hier zusätzlich noch Treiber installieren. Prinzipiell könnte man auch die Bilder mit der RPI-Bibliothek aufnehmen und sie an OpenCV geben, jedoch gibt es dann zwischen der RPI-Bibliothek und MP unlösbare Paketkonflikte.

8.3 Flugverhalten der ersten Drohne

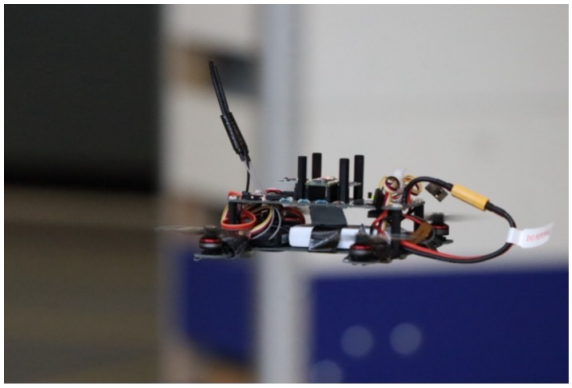


Abbildung 8.2: Erster Testflug.

Vermutlich lag das Flattern (Kapitel 7.1) am „Ground Effekt“. Dieser wird relevant, wenn ein Propeller nahe am Boden ist und vergleichsweise eine grosse Flügelfläche und -Länge hat. Durch die Nähe zum Boden kann die Luft nicht frei nach unten entweichen und staut sich unter dem Propeller auf. Dadurch entsteht mehr Auftrieb (höherer Druck auf der Unterseite bei gleichbleibendem Druck auf der Oberseite, im Vergleich zum Flug ohne den Effekt).

Wenn einer der Rotoren zufällig etwas absinkt, versucht der FC dieses Absinken zu korrigieren, indem er am absinkenden Rotor mehr Leistung gibt. Gleichzeitig generiert der abgesunkene Rotor aber auch mehr Auftrieb, weil er näher am Boden ist und somit mehr im Ground Effekt liegt, was gesamthaft zu einer Überkorrektur führt. Darauf findet der gleiche Prozess für die gegenüberliegende Seite statt und die Drohne schaukelt. Sobald aber die Drohne etwas Höhe hat, verschwindet dieser Effekt (weil auch der Ground Effekt bei genügender Höhe verschwindet) und die Drohne fliegt ruhig.

Drohnenhersteller wie DJI lösen sie dieses Problem, indem die Software die Drohnen fast nie nahe am Boden fliegen lässt. Nur beim Abheben und Landen kommen sie ganz kurz in diesen Bereich. Das kurze Durchfliegen dieses Bereichs mit Ground-Effekt reduziert die Instabilität dieser Drohnen drastisch.

8.4 Ausblick

Aufbauend auf dieser Arbeit dient die entwickelte Systemarchitektur als modulare Basis für vielfältige Applikationen. Zukünftige Anwendungsfälle, wie beispielsweise die Drohnenunterstützung bei der Rehkitzsuche, werden durch den vereinfachten Austausch des zugrundeliegenden KI-Modells wesentlich erleichtert.

Die gesetzliche Lage macht den Einsatz von Drohnen in der Schweiz grundsätzlich schwer. Vor allem, wenn es um autonome Drohnen geht. Dennoch gibt es viele Anwendungsmöglichkeiten im Schweizer Luftraum und ebenfalls viele in geschlossenen Räumen (wie es am Beispiel von Verity gezeigt wurde). Solche autonomen Drohnen können aber auch in Entwicklungsländern und Ländern mit geringerer Wohndichte eingesetzt werden,

wo sie oft auch sinnvoll sind.

Als mögliche Fortsetzung dieses Projekts könnte man beispielsweise die aktuelle Applikation verbessern oder eine neue umsetzen. Ausserdem könnte man eine, über das Internet erreichbare, graphische Benutzeroberfläche erstellen, um die Drohne zu steuern. Eine graphische Benutzeroberfläche wurde zwar am Anfang getestet, jedoch hat sie es aus Zeit und Komplexitätsgründen nicht in die Arbeit geschafft. Die Drohne selbst liesse sich ebenso verbessern und verschönern.

Mit Rückblick auf Tennekes Buch „The Simple Science of Flight“ wird klar, dass die Technik von heute weit über die einstigen *Aluminium Riesen* hinausgeht. Wir haben nun *denkende* und gewissermaßen entscheidungsfreudige Maschinen in die Luft gebracht. Diese Entwicklung führt die technische Leistungsfähigkeit immer näher an die natürliche Kognition von Tieren heran.

Literatur

[1] Henk Tennekes. *The Simple Science of Flight.*

[2] Schweizerische Rettungsflugwacht Rega. *Rega-Drohne — Schweizerische Rettungsflugwacht Rega.* de. URL: https://www.rega.ch/im-einsatz/rega-drohne (besucht am 06. 11. 2025).

[3] Welch Labs. *ChatGPT is made from 100 million of these [The Perceptron].* Feb. 2025. URL: https://www.youtube.com/watch?v=l-9ALe3U-Fg (besucht am 06. 11. 2025).

[4] *How Neural Networks Solve the XOR Problem.* en-US. Section: Artificial Intelligence. URL: https://www.geeksforgeeks.org/artificial-intelligence/how-neural-networks-solve-the-xor-problem/ (besucht am 14. 11. 2025).

[5] Aniruddha Karajgi. *How Neural Networks Solve the XOR Problem.* en-US. Nov. 2020. URL: https://towardsdatascience.com/how-neural-networks-solve-the-xor-problem-59763136bdd7/ (besucht am 17. 11. 2025).

[6] *LeNet-5 Architecture.* en-US. Section: Blogathon. URL: https://www.geeksforgeeks.org/computer-vision/lenet-5-architecture/ (besucht am 06. 11. 2025).

[7] *Bildklassifizierung mit Deep Learning: Ein umfassender Leitfaden.* de. URL: https://www.toolify.ai/de/ai-news-de/bildklassifizierung-mit-deep-learning-ein-umfassender-leitfaden-3652468 (besucht am 17. 11. 2025).

[8] *Was ist Computer Vision? — IBM.* de. Juli 2021. URL: https://www.ibm.com/de-de/think/topics/computer-vision (besucht am 17. 11. 2025).

[9] *About Verity — Pioneering AI-Powered Autonomous Mobile Intelligence Solutions.* URL: https://www.verity.net/company (besucht am 06. 11. 2025).

[10] Michael Gloor. *Verity.*

[11] Nino Florjancic und Yves Meierhans. *Multi-Sensor Drone.*

[12] Larry Li. „Time-of-Flight Camera – An Introduction". en. In: (2014).

[13] Prof. Dr. Teddy Loeliger u. a. *Compact 3D Time-of-Flight Sensor with Real-Time Multi-Path Separation.* Jan. 2025.

[14] Prof. Dr. Teddy Loeliger. *ZHAW Forschungsdrohne.*

[15] Prof. Dr. Teddy Loeliger. *SNF Research Project: 3D Time-of-Flight in Sensor Fusion.* Sep. 2024.

[16] CarbonAeronautics. *Quadcopter Manual*. en. Apr. 2023. URL: https://github.com/CarbonAeronautics/Manual-Quadcopter-Drone/blob/main/Carbon_Aeronautics_Quadcopter_Manual.pdf (besucht am 06. 11. 2025).

[17] Ravi Teja. *Brushless DC Motors (BLDC Motor): Working, Diagrams*. en-US. Sep. 2024. URL: https://www.electronicshub.org/brushless-dc-motor-bldc-motor/ (besucht am 14. 11. 2025).

[18] Joseph Flynt. *What is a Drone ESC and How Does it Work?* en-US. Apr. 2019. URL: https://3dinsider.com/drone-esc/ (besucht am 06. 11. 2025).

[19] *PWM - Pulsweitenmodulation (PDM PLM PBM)*. URL: https://www.elektronik-kompendium.de/sites/kom/0401111.htm (besucht am 06. 11. 2025).

[20] *Drone Flight Controllers: A Comprehensive Guide — Grepow*. Nov. 2024. URL: https://www.grepow.com/blog/what-is-a-drone-flight-controller.html (besucht am 06. 11. 2025).

[21] *A Complete Guide to Inertial Measurement Unit (IMU)*. en-US. Nov. 2023. URL: https://www.jouav.com/blog/inertial-measurement-unit.html (besucht am 17. 11. 2025).

[22] *Tiefpassfilter verstehen: Definition, Typen und Anwendungen — SoundScapeHQ*. de-DE. Section: Audiotechnologie. Dez. 2023. URL: https://soundscapehq.com/de/was-ist-ein-tiefpassfilter/ (besucht am 17. 11. 2025).

[23] *PID Regler · Definition & Übertragungsfunktion*. de. URL: https://studyflix.de/informatik/pid-regler-1450 (besucht am 08. 11. 2025).

[24] Jacob Stoner. *What is S.Bus (Serial Bus) & How Does it Work?* de. Okt. 2024. URL: https://www.flyeye.io/drone-acronym-sbus/ (besucht am 08. 11. 2025).

[25] *MediaPipe-Lösungsleitfaden — Google AI Edge*. de-x-mtfrom-en. URL: https://ai.google.dev/edge/mediapipe/solutions/guide?hl=de (besucht am 14. 11. 2025).

[26] *About*. en-US. URL: https://opencv.org/about/ (besucht am 17. 11. 2025).

[27] *DroneKit*. en. URL: http://dronekit.io (besucht am 14. 11. 2025).

[28] freeCodeCamp.org. *Advanced Computer Vision with Python - Full Course*. Mai 2021. URL: https://www.youtube.com/watch?v=01sAkU_NvOY (besucht am 17. 11. 2025).

[29] *OpenCV: Template Matching*. URL: https://docs.opencv.org/3.4/d4/dc6/tutorial_py_template_matching.html (besucht am 09. 11. 2025).

[30] Ivan Goncharov. *Custom Hand Gesture Recognition with Hand Landmarks Using Google's Mediapipe + OpenCV in Python*. März 2022. URL: https://www.youtube.com/watch?v=a99p_fAr6e4 (besucht am 06. 11. 2025).

[31] *Anleitung zur Erkennung von Handmarken — Google AI Edge — Google AI for Developers*. de-x-mtfrom-en. URL: https://ai.google.dev/edge/mediapipe/solutions/vision/hand_landmarker?hl=de (besucht am 15. 11. 2025).

[32] *NumPy Creating Arrays*. en-US. URL: https://www.w3schools.com/python/numpy/numpy_creating_arrays.asp (besucht am 17. 11. 2025).

[33] *Leitfaden zur Erkennung von Handmarkierungen für Python — Google AI Edge — Google AI for Developers*. de-x-mtfrom-en. URL: https://ai.google.dev/edge/mediapipe/solutions/vision/hand_landmarker/python?hl=de (besucht am 15. 11. 2025).

[34] *MediaPipe Studio*. URL: https://mediapipe-studio.webapps.google.com/demo/hand_landmarker?hl=de (besucht am 15. 11. 2025).

[35] Nikita Kiselov. *kinivi/hand-gesture-recognition-mediapipe*. original-date: 2021-01-09T21:37:23Z. Nov. 2025. URL: https://github.com/kinivi/hand-gesture-recognition-mediapipe (besucht am 06. 11. 2025).

[36] *Development Drone Kits*. en. URL: https://holybro.com/collections/multicopter-kit (besucht am 15. 11. 2025).

[37] ArduPilot. *ArduPilot*. en. URL: https://ardupilot.org (besucht am 06. 11. 2025).

[38] *Holybro*. en. URL: https://holybro.com/ (besucht am 18. 11. 2025).

[39] *PX4 Development Kit - X500 v2*. en. URL: https://holybro.com/products/px4-development-kit-x500-v2 (besucht am 18. 11. 2025).

[40] Bundesamt für Zivilluftfahrt BAZL. *Drohnen*. de. URL: https://www.bazl.admin.ch/bazl/de/home/drohnen.html (besucht am 06. 11. 2025).

[41] *Companion Computers — Dev documentation*. URL: https://ardupilot.org/dev/docs/companion-computers.html (besucht am 16. 11. 2025).

[42] Scott Campbell. *Basics of UART Communication*. en-US. Feb. 2016. URL: https://www.circuitbasics.com/basics-uart-communication/ (besucht am 16. 11. 2025).

[43] *MAVLink Developer Guide — MAVLink Guide*. URL: https://mavlink.io/en/ (besucht am 16. 11. 2025).

[44] *MAVSDK — PX4 Guide (main)*. URL: https://docs.px4.io/main/en/robotics/mavsdk (besucht am 16. 11. 2025).

[45] *MAVSDK (main / v3) — MAVSDK Guide*. URL: https://mavsdk.mavlink.io/main/en/index.html (besucht am 11. 11. 2025).

[46] google-ai-edge. *mediapipe-samples/examples/hand_landmarker/python/hand_landmarker.ipynb at main · google-ai-edge/mediapipe-samples*. en. URL: https://github.com/google-ai-edge/mediapipe-samples/blob/main/examples/hand_landmarker/python/hand_landmarker.ipynb (besucht am 16. 11. 2025).

[47] Protonest IoT. *How to Use Raspberry Pi Camera for Machine Learning with OpenCV and Picamera2*. en. Juli 2024. URL: https://protonestiot.medium.com/how-to-use-raspberry-pi-camera-for-machine-learning-with-opencv-and-picamera2-ecc663407afe (besucht am 16. 11. 2025).

Verwendung von KI

KI wurde beim Debugging des Codes verwendet (also der Suche von Tipp- und Programmierfehler im Code selbst). Dabei wurden von der KI auch weiterführende Links zu Programmierwebsites, etc. mit den entsprechenden Themen gegeben. Ausserdem wurden vereinzelt Codeblöcke von KI kopiert (vor allem bei Bash-Skripten). Code wurde vereinzelt durch KI kommentiert und von KI korrigiert. Dabei verwendet wurden:

1. ChatGPT von OpenAI
2. Claude von Anthropic
3. Gemini von Google
4. Perplexity von Perplexity
5. Gemma von Google
6. Qwen von Alibaba Cloud
7. Llama von Meta
8. Mistral von Mistral AI

Diese Liste ist umfassend. Nebenbei zu erwähnen, gilt es, dass KI ein Bestandteil der Arbeit ist.